Sveučilište Jurja Dobrile u Puli
Fakultet informatike
Preddiplomski studij informatike

Formalni jezici

skripta za pripremu ispita

"Pripada li ova riječ ovome jeziku?"

Jedno pitanje. Pet klasa jezika.
Beskonačno mnogo posljedica.

Petar Prenc

Akademska godina 2025./2026.

Pula, 27. travnja 2026.

Predgovor

Najbolji način da nešto stvarno naučiš jest da to objasniš nekom drugom. Ova skripta upravo je to: pokušaj da samom sebi — a sad i vama — na jednom mjestu objasnim što su to formalni jezici, na način koji se može pročitati u jednom dahu, bez otvaranja desetak izvora paralelno.

0.0 Što je ovo i zašto postoji

Skripta je nastala iz vrlo obične potrebe. Pripremajući se za ispit iz kolegija *Formalni jezici*, primijetio sam da je gradivo na predavanjima i u udžbeniku raspršeno: definicije ovdje, primjeri ondje, dokazi negdje između. Da bih ga svladao, morao sam ga prvo okupiti na jednom mjestu — vlastitim riječima, slijedeći jednu jasnu nit od početka do kraja. Rezultat te vježbe je tekst pred vama.

Cilj je skroman: **biti razumljiv prijateljski vodič kroz gradivo**, ništa više. Ne pretendira zamijeniti službeni udžbenik niti biti referenca posljednje riječi u svakom dokazu. Želi biti ono što bih i sam htio imati u rukama tjedan dana prije ispita — kratak, jasan, pun primjera i s naglaskom na intuiciji.

0.0 Što skripta sadrži

Materijal prati **Chomskyjevu hijerarhiju** odozdo prema gore. Krećemo od najjednostavnijih objekata — riječi i jezika kao skupova riječi — i postupno gradimo sve moćnije formalizme. Svako poglavlje prirodno se nastavlja na prethodno: regularni izrazi vode do automata, automati i izrazi pokazuju se ekvivalentnima, ekvivalencija se proteže i na linearne gramatike, a kad ti formalizmi postanu preuski, prelazimo na bezkontekstne, kontekstne i konačno Turingove strojeve.

Plan skripte

- **Poglavlje 1** — Jezici i osnovne operacije: abeceda, riječ, Σ^*, konkatenacija, Kleeneva zvijezda, Chomskyjeva hijerarhija.
- **Poglavlje 2** — Regularni izrazi i njihova algebra.
- **Poglavlje 3** — Konačni automati (KA, NKA), ε-prijelazi, partitivna kons-

trukcija.

- **Poglavlje 4** — Ekvivalentnost klasa regularnih jezika (RI, KA, NKA, DLG, LLG); Thompsonova konstrukcija, Ardenovo pravilo.
- **Poglavlje 5** — Bezkontekstni jezici, potisni automati, Chomskyjeva normalna forma, CYK algoritam.
- **Poglavlje 6** — Leme o pumpanju za regularne i bezkontekstne jezike.
- **Poglavlje 7** — Kontekstni jezici, ograničeni automati, kontekstne i monotone gramatike.
- **Poglavlje 8** — Turingovi strojevi, odlučivost, svođenje i dijagonalizacija.

0.0 Kako čitati skriptu

Tekst je organiziran tako da se može čitati linearno — svaki pojam uvodi se prije nego što se koristi. Ako već imate nešto pripreme, slobodno preskačite ono što vam je poznato; ali ako pojma nemate odakle krenuti, **počnite od početka**, najbolje s olovkom u ruci.

Vizualno je sve organizirano u obojene okvire pa se brzo prepoznaje što se gleda:

Plavo (definicije)	precizan, formalan iskaz pojma
Zeleno (primjeri)	konkretne ilustracije, "vidi i probaj"
Žuto (ključne ideje)	ono što morate ponijeti sa sobom
Ljubičasto (zadaci)	vježbe za samostalan rad

Na kraju svakog poglavlja stoji kratak **Sažetak** s tablicom najvažnijih oznaka — pogodno za brzo ponavljanje neposredno pred ispit.

0.0 O stilu i izvorima

Naglasak je na *intuiciji* i *povezanosti* pojmova, a ne na potpunoj formalnoj striktnosti svakog dokaza. Tamo gdje su dokazi tehnički zahtjevni, dane su *skice* ili upute na izvore. Cilj nije matematička virtuoznost — cilj je da, kad vam netko pokaže neki jezik, instinktivno znate razmišljati: *regularan? bezkontekstni? mogu li ga uopće prepoznati strojem?*

Glavni izvori bili su predavanja i službene bilješke kolegija, te Sipserov *Introduction to the Theory of Computation*. Eventualne pogreške isključivo su moje — ako ih primijetite, slobodno javite na petar.prenc@gmail.com.

Sretno!

Petar Prenc
Pula, 27. travnja 2026.

Sadržaj

Predgovor **1**

1 Jezici i osnovne operacije **6**
1.1 Tri pogleda na isti objekt . . . 6
1.2 Abecede i riječi . . . 6
1.3 Konkatenacija . . . 7
1.4 Jezici . . . 8
1.5 Kleeneve operacije . . . 9
1.6 Pozitivni jezici . . . 10
1.7 Chomskyjeva hijerarhija . . . 11

2 Regularni izrazi **13**
2.1 Motivacija: kako opisati beskonačni jezik? . . . 13
2.2 Definicija regularnih izraza . . . 14
2.2.1 Inicijalni jezici . . . 14
2.2.2 Operacije . . . 14
2.3 Primjeri — od jednostavnog prema složenom . . . 15
2.4 Algebarske jednakosti . . . 16
2.5 Što regularni izrazi ne mogu? . . . 17

3 Konačni automati **19**
3.1 Ideja: stroj koji čita riječ . . . 19
3.2 Deterministički konačni automat (KA) . . . 20
3.3 Nedeterministički konačni automat (NKA) . . . 21
3.3.1 Epsilon-prijelazi . . . 22
3.4 Ekvivalentnost KA i NKA . . . 22
3.4.1 Partitivna konstrukcija . . . 22
3.5 Zatvorenost na skupovne i jezicne operacije . . . 23
3.5.1 Skupovne operacije . . . 23
3.5.2 Jezicne operacije . . . 23
3.6 Kako dizajnirati KA: heuristike . . . 24

4 Ekvivalentnost klasa regularnih jezika **26**
4.1 Krug koji treba zatvoriti . . . 27
4.2 Desnolinearne gramatike (DLG) . . . 27
4.3 Pretvorba KA → DLG . . . 28

4.4 Gramaticki sustav i Ardenovo pravilo . . . 28
4.4.1 Gramaticki sustav . . . 29
4.4.2 Problem: rekurzija . . . 29
4.5 RI → NKA: Thompsonova konstrukcija . . . 29
4.6 Lijevolinearne gramatike i reverzi . . . 30
4.7 Zakljucak: sve je isto . . . 31

5 Bezkontekstni jezici **32**
5.1 Motivacija i definicija BKG . . . 32
5.2 Zatvorenost klase BK . . . 33
5.3 Potisni automati (PA) . . . 34
5.3.1 Jednostavni PA (JPA) . . . 34
5.4 Ekvivalentnost BKG i PA . . . 35
5.4.1 BKG → PA . . . 35
5.4.2 PA → BKG . . . 35
5.5 Chomskyjeva normalna forma (ChNF) . . . 36
5.5.1 Algoritam pretvorbe u ChNF (5 faza) . . . 36
5.6 Algoritam CYK . . . 37
5.6.1 Ideja: dinamicko programiranje . . . 37
5.6.2 Tablica i algoritam . . . 37
5.6.3 Rekonstrukcija stabla parsiranja . . . 38
5.7 Stabla parsiranja i viseznacanost . . . 38

6 Leme o pumpanju **40**
6.1 Lema o pumpanju za regularne jezike . . . 40
6.2 Primjena: $L_=$ nije regularan . . . 41
6.3 Lema o pumpanju za bezkontekstne jezike . . . 42
6.4 Primjena: $L_\equiv$ nije bezkontekstan . . . 42
6.5 Usporedba dviju lema . . . 43

7 Kontekstni jezici **45**
7.1 Motivacija: sto ne moze BK? . . . 45
7.2 Ograniceni automat (OA) . . . 45
7.2.1 Odlucivost A_{OA} . . . 46
7.3 Poboljsanja OA . . . 46
7.4 Kontekstne i monotone gramatike . . . 47
7.4.1 Monotona gramatika za $L_\equiv$ . . . 47
7.4.2 Pretvorba MG → KG . . . 48
7.5 Ekvivalentnost OA i MG . . . 48
7.5.1 MG → OA . . . 48
7.5.2 OA → MG . . . 48
7.6 Zatvorenost klase Kon . . . 48
7.7 Usporedba klasa . . . 49

8 Turingovi strojevi i odlucivost **51**
8.1 Turingov stroj — motivacija . . . 51
8.2 Turingov prepoznavac (TP) . . . 52

8.3 Turingov odlucitelj (TO) 52
8.4 Zatvorenost klasa Rek i RE 53
8.5 Kodiranje i problemi 53
8.5.1 Prirodni strojevi i problemi 53
8.6 Odlucivi i neodlucivi problemi 53
8.6.1 Odlucivi problemi 53
8.6.2 Neodlucivi problemi — hijerarhija 54
8.7 Svodenje (redukcija) 54
8.8 Dijagonalizacija: Russell i A_{TP} 55
8.9 Problem zaustavljanja i kompletna slika 55
8.9.1 Kompletna Chomskyjeva hijerarhija 56
8.10 Nedeterministicki Turingov stroj (NTP) 56
8.11 Turingovi nabrajaci (TE) 57

Poglavlje 1

Jezici i osnovne operacije

Sve što ćemo raditi u ovom kolegiju vrti se oko jednog jednostavnog pitanja: pripada li neka riječ nekom jeziku? Ovo poglavlje postavlja temelje — definira što su uopće „riječi" i „jezici", i uvodi osnovne operacije s kojima ćemo ih graditi i kombinirati.

1.1 Tri pogleda na isti objekt

Zamislite funkciju $f : \mathbb{R} \to \mathbb{R}$. Možemo je opisati na barem tri načina: kao skup uređenih parova $\{(x, f(x)) \mid x \in \mathbb{R}\}$, kao formulu poput $f(x) = x^2 + 1$, ili kao program koji za svaki ulaz x vraća odgovarajući izlaz. Sva tri opisa govore o istoj stvari, ali svaki naglašava nešto drugačije i koristan je u različitim situacijama.

Formalni jezici bave se istim trikom, ali za jezike umjesto funkcija. **Jezik** je skup — statični popis riječi koje mu pripadaju. **Gramatika** je dinamično pravilo koje taj skup *generira*, kreće od početnog simbola i primjenjuje pravila prepisivanja. **Automat** je idealizirani stroj koji *prepoznaje* jezik čitajući ulaznu riječ znak po znak i na kraju odlučuje: „da" ili „ne".

Ono što čini ovaj predmet zanimljivim jest to da između ova tri pogleda postoje duboke i neočite veze. Na primjer, nije odmah jasno zašto bi za svaku gramatiku određenog tipa *nužno* postojao automat koji prepoznaje isti jezik — ali to je upravo ono što ćemo dokazivati.

Sve klase koje ćemo proučavati organizirane su u **Chomskyjevu hijerarhiju**, kojoj se vraćamo na kraju poglavlja.

1.2 Abecede i riječi

Prije nego što možemo govoriti o jezicima, moramo dogovoriti s kojim simbolima radimo.

Definicija 1.2.1 — Abeceda

Abeceda je konačni, neprazni skup Σ. Elemente abecede zovemo **znakovima** i često označavamo malim grčkim slovima: $\alpha, \beta, \gamma, \ldots$

Ništa posebno — abeceda je samo dogovor o tome koji su nam simboli na raspolaganju. Ograničenje da mora biti *konačna* je ključno: svi naši strojevi i gramatike imat će konačni opis, pa moramo krenuti od konačne abecede.

Primjer 1.2.1 — Nekoliko uobičajenih abeceda

U praksi ćemo najčešće raditi s:

- $\Sigma = \{0, 1\}$ — binarna abeceda, dovoljna za kodiranje čega god želimo
- $\Sigma = \{a, b\}$ ili $\Sigma = \{a, b, c\}$ — mali primjeri za ilustracije
- $\Sigma = \{0, 1, \ldots, 9\}$ — decimalne znamenke

Abeceda ne mora biti skup slova ili znamenki — može biti skup tokena, boja, instrukcija ili bilo čega drugog. Bitno je samo da je konačna.

Definicija 1.2.2 — Riječ i njezina duljina

Riječ nad abecedom Σ je konačni niz znakova iz Σ:

$$w = \alpha_1 \alpha_2 \cdots \alpha_n, \qquad \alpha_i \in \Sigma.$$

Duljina riječi w, pisana $|w|$, jest broj znakova u njoj. Pišemo također $|w|_\alpha$ za broj pojavljivanja znaka α u w.

Posebno mjesto zauzima **prazna riječ** ε — jedina riječ duljine 0. To nije znak, nego prazan niz znakova, i vrijedi $|\varepsilon| = 0$.

Napomena. *Prazna riječ ε i prazan* jezik $\emptyset$ *nisu isto. Prazna riječ* postoji *kao objekt, samo ne sadrži nijedan znak. Prazan jezik je skup koji ne sadrži nijednu riječ — pa onda ni ε. Ovu razliku je lako zamijeniti na ispitu, pazite.*

Uobičajeno je poistovjećivati znak $\alpha \in \Sigma$ s jednakovrijednom jednoznakovnom riječi. Dakle, a kao znak i a kao jednoelementna „rečenica" su isti objekt — samo gledamo iz drugog kuta.

1.3 Konkatenacija

Osnovna operacija kojom iz kratkih riječi gradimo dulje jest **konkatenacija** — spajanje dvije riječi jedne za drugom.

Definicija 1.3.1 — Konkatenacija

Za riječi $u = \alpha_1 \cdots \alpha_n$ i $v = \beta_1 \cdots \beta_m$ definiramo

$$uv \;:=\; \alpha_1 \cdots \alpha_n \, \beta_1 \cdots \beta_m.$$

Vrijedi $|uv| = |u| + |v|$ — duljine se zbrajaju.

Konkatenacija je asocijativna: $(uv)w = u(vw)$, pa zagrade su suvišne i pišemo jednostavno uvw. Prazna riječ ε je neutralni element: $\varepsilon w = w\varepsilon = w$ — dodavanje ničega ništa ne mijenja.

Budući da konkatenacija „izgleda kao množenje", koristimo uobičajenu multiplikativnu notaciju za potencije:

$$u^0 := \varepsilon, \qquad u^1 := u, \qquad u^2 := uu, \qquad u^3 := uuu, \quad \ldots$$

Primjer 1.3.1

Neka je $u = ab$ i $v = ba$. Tada $uv = abba$, $vu = baab$, $u^3 = ababab$. Konkatenacija nije komutativna — $uv \neq vu$ u općem slučaju.

1.4 Jezici

Sad kad znamo što su riječi, možemo definirati jezik.

Definicija 1.4.1 — Σ^* i jezik

Skup *svih* mogućih riječi nad abecedom Σ (uključujući ε) označavamo s Σ^*:

$$\Sigma^* \;:=\; \{\varepsilon\} \cup \Sigma \cup \Sigma^2 \cup \Sigma^3 \cup \cdots$$

Jezik nad Σ je bilo koji podskup $L \subseteq \Sigma^*$.

Definicija je namjerno vrlo široka — jezik je doslovno *bilo koji* skup riječi, konačni ili beskonačni. Jedino ograničenje je da su sve riječi nad istom abecedom. Nema uvjeta da jezik mora imati neku posebnu strukturu; to tek postaje zanimljivo kad počnemo klasificirati jezike po tome *kakvu* strukturu imaju.

Primjer 1.4.1

Nad $\Sigma = \{a, b\}$, evo nekoliko jezika raznih vrsta:

- $L_1 = \{a, ab, aba\}$ — konačni jezik s tri elementa
- $L_2 = \{w \in \Sigma^* \mid |w|_a = |w|_b\}$ — sve riječi s jednakim brojem a-ova i b-ova; beskonačni
- $L_3 = \{a^n b^n \mid n \geq 1\} = \{ab, aabb, aaabbb, \ldots\}$ — klasični primjer koji ćemo često susretati
- $\emptyset$ — prazan jezik; nema nijedne riječi
- $\{\varepsilon\}$ — jezik s jednom riječi, praznom
- Σ^* — svi mogući nizovi znakova, bez ikakva ograničenja

Jezike možemo kombinirati skupovnim operacijama ($\cup$, $\cap$, $\setminus$, komplement spram Σ^*), ali i operacijom konkatenacije, proširenom s riječi na skupove:

Definicija 1.4.2 — Konkatenacija jezika

Za jezike $L, M \subseteq \Sigma^*$ definiramo

$$LM \;:=\; \{uv \mid u \in L,\; v \in M\}.$$

Dakle, LM se dobiva lijepljenjem svake riječi iz L sa svakom riječi iz M.

Primjer 1.4.2

Neka je $L = \{a, ab\}$ i $M = \{b, c\}$. Sve kombinacije su:

$$LM = \{a \cdot b,\; a \cdot c,\; ab \cdot b,\; ab \cdot c\} = \{ab,\; ac,\; abb,\; abc\}.$$

A što je $L^2 = LL$? Svaka riječ iz L zalijepljena sa svakom:

$$L^2 = \{aa,\; aab,\; aba,\; abab\}.$$

Primijetite da L^2 ima točno $|L|^2 = 4$ elementa — u ovom slučaju nema „sudara" (različitih kombinacija koje daju istu riječ). Nije uvijek tako.

1.5 Kleeneve operacije

Najvažnija operacija u cijelom kolegiju je Kleeneva zvijezda. Susresti ćete je svugdje — u regularnim izrazima, gramatikama, posvuda.

Definicija 1.5.1 — Kleenejev plus i zvijezda

Za jezik $L \subseteq \Sigma^*$ definiramo:

$$L^+ \;:=\; \bigcup_{k \geq 1} L^k \;=\; L \cup L^2 \cup L^3 \cup \cdots$$

$$L^* \;:=\; \bigcup_{k \geq 0} L^k \;=\; \{\varepsilon\} \cup L \cup L^2 \cup L^3 \cup \cdots$$

Razlika je samo u tome što L^* uključuje $L^0 = \{\varepsilon\}$, a L^+ ne. Vrijedi $L^* = \{\varepsilon\} \cup L^+$, tj. zvijezda je plus uz dodatak prazne riječi.

Kako čitati ove definicije? L^* je skup svih *nizanja* nula ili više riječi iz L jedne za drugom. Kad kažemo da neka riječ w pripada L^*, mislimo da se w može „izrezati" na komadiće pri čemu svaki komadić pripada L.

Ključna ideja

$w \in L^*$ **ako i samo ako** postoji rastav $w = w_1 w_2 \cdots w_k$ (za neko $k \geq 0$) takav da $w_i \in L$ za svaki i. Slučaj $k = 0$ daje $w = \varepsilon$ — uvijek prihvatljivo.

Primjer 1.5.1

Uzmimo $L = \{0, 11\}$ nad $\Sigma = \{0, 1\}$. Tada:

$$L^* = \{\varepsilon,\ 0,\ 11,\ 00,\ 011,\ 110,\ 1111,\ 000,\ \ldots\}$$

Je li $110011 \in L^*$? Da — rastav je $\underbrace{11}\ \underbrace{0}\ \underbrace{0}\ \underbrace{11}$, svaki komadić je u L.

Je li $1 \in L^*$? Ne. Jedina jedinka u L je 0; jedini dvoznakac je 11; nema načina sastaviti samu 1 od komadića iz $\{0, 11\}$.

Primjer 1.5.2

Posebno važan slučaj: ako je Σ abeceda (shvaćena kao jezik jednoznakovnih riječi), tada je Σ^* upravo *skup svih mogućih riječi* nad Σ — što je i definicija iz prethodnog odjeljka. Zvijezda u oznaci Σ^* nije slučajnost.

Postoji korisna analogija s aritmetikom. Konkatenacija je množenje, prazna riječ ε je jedinica, a Kleeneva zvijezda nalikuje na „beskonačnu geometrijsku sumu" $1 + L + L^2 + \cdots$, samo u algebri jezika. Ta analogija nije samo poetska — na njoj se temelji teorija regularnih izraza koju ćemo graditi u sljedećem poglavlju.

Zadatak 1.5.1

(a) Izrazite L^* pomoću L^+ i obrnuto. Koji uvjet mora biti ispunjen da vrijedi $L^+ = L^* \setminus \{\varepsilon\}$?

(b) Dokažite ili opovrgnite: $(L \cup M)^* = (L^* M^*)^*$.

(c) Pokažite da je $\emptyset^* = \{\varepsilon\}$. Intuitivno, zašto to ima smisla?

(d) Za $L = \{a\}$, opišite L^* riječima. Koji je to dobro poznati skup?

1.6 Pozitivni jezici

Jedno naizgled sitno, ali tehnički korisno svojstvo jest pozitivnost.

Definicija 1.6.1 — Pozitivan jezik

Kažemo da je jezik L **pozitivan** ako $\varepsilon \notin L$.

Zašto nam je to uopće važno? U dokazima koji idu indukcijom po duljini riječi, pozitivnost osigurava da konkatenacijom ne možemo ostati na istoj duljini. Precizno: ako je L pozitivan i $w \in LM$, dio koji pripada M strogo je kraći od w — pa indukcija „napreduje".

Vrijedi nekoliko jednostavnih pravila:

- $\emptyset$ je pozitivan (trivijalno — nema elemenata, pa ni ε).
- Svaki jednoznakovni jezik $\{\alpha\}$ je pozitivan.

- L^* *nikad nije* pozitivan — uvijek vrijedi $\varepsilon \in L^*$.
- LM je pozitivan ako i samo ako je barem jedan od L, M pozitivan.
- $L \cup M$ je pozitivan ako i samo ako su *oba* L i M pozitivna.

Zadnja dva pravila su dobar test razumijevanja — probajte ih dokazati sami iz definicija.

1.7 Chomskyjeva hijerarhija

Na kraju poglavlja smjestimo sve u širi okvir. Chomskyjeva hijerarhija klasifikacija je jezika u klase prema tome koliko su „teški za prepoznati".

Neformalna ideja: što je jezik strukturiraniji i zahtjevniji, to je moćniji stroj (ili gramatika) potreban za njegovo opisivanje. Najjednostavniji jezici — **regularni** — mogu se prepoznati *bez ikakva pamćenja* osim trenutnog stanja. Za malo zahtjevnije — **bezkontekstne** — treba nam stog (neograničena memorija, ali pristup samo s vrha). Zatim dolaze **kontekstni** jezici koji trebaju traku jednake duljine kao ulaz. Na samom vrhu su **rekurzivno prebrojivi** jezici, za koje treba puni Turingov stroj.

Svakoj klasi odgovara vrsta gramatike i vrsta automata:

Klasa	**Oznaka**	**Gramatika**	**Automat**
Regularni	Reg	desno/lijevolinearna	konačni (KA, NKA)
Bezkontekstni	BK	bezkontekstna (BKG)	potisni (PA)
Kontekstni	Kon	kontekstna (KG)	ograničeni (OA)
Rekurzivni	Rek	—	Turingov odlučitelj
Rek. prebrojivi	RE	opća (OG)	Turingov prepoznavač

Među klasama vrijede *prave inkluzije* — svaka klasa je pravi podskup sljedeće:

$$\mathrm{Reg} \subsetneq \mathrm{BK} \subsetneq \mathrm{Kon} \subsetneq \mathrm{Rek} \subsetneq \mathrm{RE}.$$

„Prava" inkluzija znači da za svaki par susjednih klasa postoji jezik koji pripada gornjoj, ali ne i donjoj. Klasični primjeri koji „razdvajaju" klase su:

$$L_= = \{a^n b^n \mid n \geq 1\} \in \mathrm{BK} \setminus \mathrm{Reg},$$
$$L_\equiv = \{a^n b^n c^n \mid n \geq 1\} \in \mathrm{Kon} \setminus \mathrm{BK}.$$

Zašto, primjerice, $L_=$ nije regularan? Intuitivno: kad konačni automat čita $a^n b^n$, u trenutku kad dođe do prvih b-ova mora nekako „znati" koliko je a-ova pročitao — a ima samo konačno mnogo stanja, pa ne može pamtiti proizvoljan broj. Formalni dokaz dat ćemo u poglavlju o lemi o pumpanju.

Što nas čeka

Kolegij prolazi kroz hijerarhiju odozdo prema gore: regularni jezici (poglavlja 2–4), bezkontekstni (poglavlje 5), kontekstni (poglavlje 7), i na kraju Turingovi strojevi i odlučivost (poglavlje 8). Svaki put kad pređemo u višu klasu, pronaći ćemo jezik koji ne stane u onu ispod — i naučiti *zašto*.

1.7 Sažetak

Ovim smo postavili temelje. U jednoj rečenici: jezik je skup riječi nad nekom abecedom, a ovo poglavlje definira kako se s takvim skupovima radi.

Σ	abeceda — konačni, neprazni skup znakova
w, ε	riječ, prazna riječ
$\lvert w \rvert$	duljina riječi
Σ^*	skup svih riječi nad Σ
$L \subseteq \Sigma^*$	jezik
LM	konkatenacija jezika
L^*, L^+	Kleeneva zvijezda i plus

U sljedećem poglavlju uvodimo **regularne izraze** — kompaktan formalizam za opisivanje regularnih jezika koji koristi upravo operacije $\cup$, konkatenaciju i * koje smo ovdje upoznali.

Poglavlje 2

Regularni izrazi

U prvom poglavlju naučili smo što su jezici i kako ih kombinirati. Sad postavljamo pitanje: kako ih opisati*? Regularni izrazi daju nam kompaktan, elegantan formalizam koji bi trebao izgledati poznato — vjerojatno ste već koristili* `grep` *ili* `re` *modul u Pythonu. Ovo poglavlje tu intuiciju pretvara u preciznu matematiku.*

2.1 Motivacija: kako opisati beskonačni jezik?

Zamislite da vam netko kaže: „napiši mi sve valjane e-mail adrese". Odmah je jasno da taj skup ne možete nabrojati — beskonačni je i nema gornje granice duljine. No ipak intuitivno znate što je valjana adresa: neka slova, pa `@`, pa domena. Regularni izrazi su upravo alat koji takav opis pretvara u preciznu matematičku definiciju.

U Pythonu ste možda pisali nešto poput `re.match(r'[a-z]+@[a-z]+\.[a-z]+', s)`. Formalni regularni izrazi koje ćemo definirati su strožija i čišća verzija iste ideje — bez šarenih proširenja koja praktični jezici dodaju, ali s jasnom matematičkom semantikom.

Ideja je jednostavna: regularni izrazi grade se od malih komadića (*inicijalnih* ili *atomarnih* jezika) kombiniranjem s tri operacije — unije, konkatenacije i Kleeneve zvijezde. Ništa više. Pokazat će se da je to i više nego dovoljno za cijelu klasu regularnih jezika.

2.2 Definicija regularnih izraza

Definicija 2.2.1 — RI-regularni jezik

Neka je Σ abeceda. **Skup RI-regularnih jezika** nad Σ je *najmanji* skup jezika koji:

- sadrži $\emptyset$ i $\{\varepsilon\}$ kao elemente,
- sadrži $\{\alpha\}$ za svaki znak $\alpha \in \Sigma$,
- je zatvoren na konkatenaciju, konačnu uniju i Kleeneve operacije.

Jezik koji pripada tom skupu zovemo **RI-regularnim**. **Regularni izraz** je izraz koji pokazuje kako je taj jezik izgrađen od navedenih operacija.

Dakle, regularni izraz nije sam po sebi jezik — on je *recept* za gradnju jezika. Kao što izraz $2 \cdot 3 + 1$ nije broj već opis kako doći do broja 7, regularni izraz ab^* nije skup nego opis kako doći do skupa $\{a, ab, abb, abbb, \ldots\}$.

2.2.1 Inicijalni jezici

Gradivni blokovi su sljedeća tri tipa:

Izraz	Jezik	Opis
$\emptyset$	$\emptyset$	prazan jezik, nema nijedne riječi
ε	$\{\varepsilon\}$	samo prazna riječ
α	$\{\alpha\}$	samo jednoznakovna riječ od α

Napomena. *Pažnja na oznake: $\emptyset$ kao regularni izraz opisuje* prazan jezik*, a ε kao regularni izraz opisuje jezik $\{\varepsilon\}$ koji sadrži* jednu *riječ. Ovo je ista razlika kao u prethodnom poglavlju — samo sad gledamo izraze, ne skupove.*

2.2.2 Operacije

Iz inicijalnih jezika gradimo veće regularnim operacijama:

Izraz	Jezik	Čitamo
$r \cup s$	$L(r) \cup L(s)$	unija: ili r ili s
$r \cdot s$	$L(r) \cdot L(s)$	konkatenacija: prvo r, pa s
r^*	$L(r)^*$	zvijezda: nula ili više puta r
r^+	$L(r)^+$	plus: jedanput ili više puta r

Točka za konkatenaciju često se izostavlja: rs znači isto što i $r \cdot s$. Prioritet operacija je kao u aritmetici: * i $^+$ vežu najjače, pa konkatenacija, pa unija. Dakle $ab^*c \cup d$ čitamo kao $(a \cdot (b^*) \cdot c) \cup d$.

2.3 Primjeri — od jednostavnog prema složenom

Sad kad imamo definiciju, vježbamo je na konkretnim primjerima. Ključna vještina je dvojaka: čitati regularni izraz i razumjeti koji jezik opisuje, i zadani jezik opisati regularnim izrazom.

Primjer 2.3.1 — Jednostavni izrazi

Nad $\Sigma = \{a, b\}$:

- $a \cup b$ opisuje jezik $\{a, b\}$ — jedna od ta dva slova.
- ab opisuje $\{ab\}$ — točno ta jedna dvoznakovna riječ.
- a^* opisuje $\{\varepsilon, a, aa, aaa, \ldots\}$ — nula ili više a-ova.
- a^+ opisuje $\{a, aa, aaa, \ldots\}$ — jedan ili više a-ova.
- a^*b opisuje $\{b, ab, aab, aaab, \ldots\}$ — nula ili više a-ova, pa jedan b.

Primjer 2.3.2 — Sve riječi nad $\{a, b\}$

Izraz $(a \cup b)^*$ opisuje *sve* moguće riječi nad $\{a, b\}$, dakle Σ^*. Kleeneva zvijezda od $\{a\} \cup \{b\} = \{a, b\}$ daje sve načine nizanja a-ova i b-ova, bez ikakva ograničenja. To je korisna crtica: kad god trebate „bilo što od tih znakova", pišete $(a \cup b)^*$, ili općenito Σ^*.

Primjer 2.3.3 — Binarni brojevi

Razmotrimo izraz $1(0 \cup 1)^* \cup 0$ nad $\Sigma = \{0, 1\}$.

- $1(0 \cup 1)^*$: počinje s 1, a zatim slijedi bilo što — to su svi binarni brojevi ≥ 1.
- 0: baš sama nula.
- Unija: skupa dobivamo *sve valjane binarne zapise prirodnih brojeva* bez vodećih nula (jedino 0 počinje s nulom).

Ovo je tipičan uzorak: posebnim slučajem (0) riješimo rub, a glavnom granom $(1(0 \cup 1)^*)$ pokrijemo opći slučaj.

Primjer 2.3.4 — Riječi koje počinju i završavaju istim slovom

Napišimo regularni izraz za sve riječi nad $\{a, b, c\}$ koje počinju i završavaju istim slovom. Koristimo kraticu $\Sigma := a \cup b \cup c$.
Fiksiramo koje je to slovo i ujedinimo tri slučaja:

- Počinje i završava s a: treba $a\Sigma^*a$, ali i samu riječ a. Dakle: $a(\Sigma^*a \cup \varepsilon)$.
- Analogno za b i c.

Ukupno:

$$a(\Sigma^*a \cup \varepsilon) \;\cup\; b(\Sigma^*b \cup \varepsilon) \;\cup\; c(\Sigma^*c \cup \varepsilon).$$

Primjer 2.3.5 — Neparan broj a-ova

Jezik svih riječi nad $\{a, b\}$ s neparnim brojem a-ova.

Intuicija: između a-ova može biti koliko god b-ova. Pokušajmo $b^*(ab^*)^+$ — ali to dopušta *bilo koji* broj a-ova, parni ili neparni.
Rješenje: svaki par a-ova se „poništi", a jedan ostaje:

$$b^*\big(ab^*ab^*\big)^*ab^*.$$

Čitanje: nula ili više parova $a \cdots a$ (okruženih b-ovima), pa točno jedan a, pa b-ovi. Ukupan broj a-ova je $2k+1$ za neko $k \geq 0$ — uvijek neparan.

Ključna ideja

Izgradnja regularnog izraza za zadani jezik obično ide ovako:

1. Identificirajte strukturu riječi — što dolazi na kojoj poziciji.
2. Podijelite na slučajeve ako treba (npr. različiti početni znakovi).
3. Koristite $(a \cup b \cup \ldots)^*$ za „bilo što od tih znakova".
4. Provjerite rubne slučajeve: prazna riječ, jednoznakovne riječi.

2.4 Algebarske jednakosti

Dva regularna izraza r i s su **ekvivalentni** ako opisuju isti jezik: $L(r) = L(s)$. Pišemo $r \equiv s$.

Postoji niz korisnih algebarskih identiteta:

Identitet	**Što kaže**
$r \cup \emptyset \equiv r$	unija s praznim jezikom ne mijenja ništa
$r \cdot \varepsilon \equiv r$	konkatenacija s ε ne mijenja ništa
$r \cdot \emptyset \equiv \emptyset$	konkatenacija s praznim daje prazno
$r \cup r \equiv r$	unija sa samim sobom je idempotentna
$r \cup s \equiv s \cup r$	unija je komutativna
$(rs)t \equiv r(st)$	konkatenacija je asocijativna
$r(s \cup t) \equiv rs \cup rt$	distributivnost
$(r^*)^* \equiv r^*$	zvijezda od zvijezde je zvijezda
$\varepsilon^* \equiv \varepsilon$	zvijezda od ε je ε
$\emptyset^* \equiv \varepsilon$	zvijezda od praznog skupa je $\{\varepsilon\}$

Primjer 2.4.1

Provjerimo $\emptyset^* \equiv \varepsilon$, tj. $L(\emptyset^*) = \{\varepsilon\}$. Po definiciji, $\emptyset^* = \bigcup_{k \geq 0} \emptyset^k$. Vrijedi $\emptyset^0 = \{\varepsilon\}$ i $\emptyset^k = \emptyset$ za $k \geq 1$. Dakle $\emptyset^* = \{\varepsilon\}$. Intuitivno: jedini način da se „niže nula ili više riječi iz praznog skupa" je uzeti nula riječi, što daje ε.

Zadatak 2.4.1

(a) Pokažite da je $r^+ \equiv rr^* \equiv r^*r$.

(b) Vrijedi li $r^* \equiv r^+ \cup \varepsilon$ za svaki r? Što ako je $\varepsilon \in L(r)$?

(c) Nad $\Sigma = \{0, 1\}$, napišite regularni izraz za:
- sve riječi koje završavaju na 01,
- sve riječi koje sadrže podniz 11,
- sve riječi parne duljine.

(d) Nad $\Sigma = \{a, b\}$, napišite regularni izraz za sve riječi u kojima se a nikad ne pojavljuje dva puta zaredom.

2.5 Što regularni izrazi ne mogu?

Regularni izrazi su moćni, ali nisu svemoćni.

Jezik $L_= = \{a^n b^n \mid n \geq 1\}$ sadrži riječi ab, $aabb$, $aaabbb$, ... Svaka riječ ima isti broj a-ova i b-ova. Pokušajte napisati regularni izraz za taj jezik — neće ići.

Zašto? Regularni izraz ne može „prebrojati" i usporediti duljine dva dijela riječi. Sve što može je pratiti konačno mnogo mogućih „stanja" — a mogućih vrijednosti n je beskonačno mnogo. Formalni dokaz dolazi s **lemom o pumpanju** (poglavlje 6), ali intuicija je jasna već sad.

Ključna ideja

Regularni izrazi opisuju jezike u kojima se struktura može pratiti bez pamćenja neograničenih brojača. Čim trebate „zapamtiti" koliko puta ste nešto vidjeli i usporediti s nekim drugim brojem — regularni izraz nije dovoljan alat.

2.5 Sažetak

Regularni izrazi daju nam kompaktan način opisivanja regularnih jezika. Grade se od tri atomarna slučaja ($\emptyset$, ε, jednoznakovni jezici) i tri operacije (unija, konkatenacija, Kleeneva zvijezda/plus).

$\emptyset$	prazan jezik
ε	jezik $\{\varepsilon\}$
α	jezik $\{\alpha\}$
$r \cup s$	unija
rs	konkatenacija
r^*	Kleeneva zvijezda (nula ili više puta)
r^+	Kleenejev plus (jedanput ili više)

U sljedećem poglavlju uvodimo **konačne automate** — strojeve koji prepoznaju regularne jezike. Vidjet ćemo da su automati i regularni izrazi *ekvivalentni*: svaki regularni izraz daje automat, i svaki automat daje regularni izraz.

Poglavlje 3

Konačni automati

Regularni izrazi opisuju što jezik sadrži — ali kako provjeriti pripada li konkretna riječ nekom jeziku? Za to trebamo stroj. Konačni automat je najjednostavniji takav stroj: čita riječ znak po znak i na kraju odlučuje da ili ne, bez ikakve memorije osim trenutnog stanja. Ovaj jednostavni model ispostavlja se jednako moćnim kao i regularni izrazi.

3.1 Ideja: stroj koji čita riječ

Zamislite jednostavan sustav za provjeru PIN-a bankomata. On prima niz znamenki i na kraju kaže: točno ili pogrešno. U svakom trenutku pamti samo "koliko smo znakova dosad prihvatili" — nema nikakve druge memorije. Takav sustav je, u biti, konačni automat.

Formalno, konačni automat je idealizirani matematički stroj koji:

- ima konačno mnogo **stanja** (kao moguće "faze" rada),
- kreće iz fiksnog **početnog stanja**,
- za svaki pročitani znak prelazi u novo stanje prema **funkciji prijelaza**,
- na kraju prihvaća riječ ako je završio u nekom od **završnih stanja**.

Kljucno ogranicenje: nema stoga, nema trake, nema ekstre memorije. Jedino sto stroj "zna" jest u kojem je stanju — i to je sve.

3.2 Deterministički konačni automat (KA)

Definicija 3.2.1 — Konačni automat

Konačni automat (KA) nad abecedom Σ je uređena petorka

$$M = (Q,\, \Sigma,\, \delta,\, q_0,\, F),$$

gdje je:

- Q — konačni skup **stanja**,
- Σ — abeceda (konačni skup ulaznih znakova),
- $\delta : Q \times \Sigma \to Q$ — **funkcija prijelaza**,
- $q_0 \in Q$ — **početno stanje**,
- $F \subseteq Q$ — skup **završnih stanja**.

Funkcija prijelaza δ je srce automata: za svako stanje i svaki ulazni znak precizno govori u koje stanje automat prelazi. Nema nedefiniranih prijelaza, nema izbora — za svaki par (q, α) postoji točno jedan nasljednik $\delta(q, \alpha)$.

Definicija 3.2.2 — Prihvaćanje riječi

KA $M = (Q, \Sigma, \delta, q_0, F)$ **prihvaća** riječ $w = \alpha_1\alpha_2 \cdots \alpha_n \in \Sigma^*$ ako postoji niz stanja $r_0, r_1, \ldots, r_n \in Q$ takav da:

- $r_0 = q_0$ (kreće iz početnog stanja),
- $\delta(r_{i-1}, \alpha_i) = r_i$ za sve $1 \leq i \leq n$ (prati prijelaze),
- $r_n \in F$ (završava u završnom stanju).

Jezik koji M prepoznaje je

$$L(M) := \{w \in \Sigma^* : M \text{ prihvaća } w\}.$$

Napomena. *Za deterministički KA, niz stanja $r_0, r_1, \ldots, r_n$ je* jedinstven *za zadane M i w — nema izbora ni slučajnosti. Automat uvijek "zna" što radi.*

Primjer 3.2.1 — KA koji prihvaća riječi s parnim brojem a-ova

Nad $\Sigma = \{a, b\}$, konstruirajmo KA za jezik svih riječi s parnim brojem pojavljivanja slova a (uključujući 0).
Trebamo pratiti samo parnost broja a-ova — dvije mogucnosti: "parno" i "neparno". To su nase dvije stanice:

$$Q = \{q_{\text{par}},\, q_{\text{nepar}}\}, \quad q_0 = q_{\text{par}}, \quad F = \{q_{\text{par}}\}.$$

Funkcija prijelaza:

	a	b
q_{par}	q_{nepar}	q_{par}
q_{nepar}	q_{par}	q_{nepar}

Riječ $aabba$: $q_{\text{par}} \xrightarrow{a} q_{\text{nepar}} \xrightarrow{a} q_{\text{par}} \xrightarrow{b} q_{\text{par}} \xrightarrow{b} q_{\text{par}} \xrightarrow{a} q_{\text{nepar}}$. Završava u $q_{\text{nepar}} \notin F$ — riječ nije prihvaćena (ima 3 slova a). ✓

Primjer 3.2.2 — KA koji prihvaća riječi koje završavaju na ab

Nad $\Sigma = \{a, b\}$, želimo prepoznati sve riječi koje završavaju nizom ab. Automat mora pamtiti "suffix koji smo dosad vidjeli", ali samo onoliko koliko je potrebno:

$$Q = \{q_0,\, q_a,\, q_{ab}\}, \quad F = \{q_{ab}\}.$$

	a	b
q_0	q_a	q_0
q_a	q_a	q_{ab}
q_{ab}	q_a	q_0

Stanje q_a znači: "zadnji pročitani znak je bio a". Stanje q_{ab}: "zadnja dva znaka su bila ab". Iz q_{ab}, ako pročitamo a, idemo u q_a (novi potencijalni početak); ako pročitamo b, gubimo trag i vraćamo se u q_0.

3.3 Nedeterministički konačni automat (NKA)

Deterministički KA je konceptualno jasan, ali često nespretan za dizajnirati. Prirodniji pristup je često *nedeterminističan*: u svakom trenutku automat smije "pogoditi" koji će prijelaz napraviti, a prihvaća riječ ako *postoji barem jedna grana* koja vodi u završno stanje.

Definicija 3.3.1 — Nedeterministički KA

Nedeterministički konačni automat (NKA) nad Σ je uređena petorka $(Q, \Sigma, \Delta, q_0, F)$, gdje je sve kao kod KA osim sto je funkcija prijelaza zamijenjena **relacijom prijelaza**:

$$\Delta \subseteq Q \times \Sigma_\varepsilon \times Q,$$

gdje je $\Sigma_\varepsilon := \Sigma \cup \{\varepsilon\}$. Prijelaz $(q, \alpha, p) \in \Delta$ znači: "iz stanja q, čitajući znak α (ili, ako je $\alpha = \varepsilon$, bez čitanja), može se preći u stanje p".
NKA prihvaća riječ w ako postoji *barem jedan* način čitanja w koji vodi iz q_0 u neko završno stanje.

Ključna ideja

Nedeterminizam je matematički trik: NKA "pogađa" točnu granu. U stvarnosti takav stroj ne postoji — ali matematički je koristan jer je često mnogo lakši za dizajnirati od ekvivalentnog KA. Ključni rezultat: svaki NKA ima ekvivalentni KA!

3.3.1 Epsilon-prijelazi

Posebno su korisni **ε-prijelazi**: automat prelazi iz jednog stanja u drugo *bez čitanja ijednog znaka*. Ovo se čini kao varanje, ali zapravo samo kaže: "automat smije besplatno proći kroz neka stanja".

Definicija 3.3.2 — ε-ljuska

Za skup stanja $S \subseteq Q$, **ε-ljuska** od S je skup svih stanja dostupnih iz S isključivo ε-prijelazima:

$$[S]_\varepsilon := \{q' \in Q : \text{postoji staza } \varepsilon\text{-prijelaza od nekog } q \in S \text{ do } q'\}.$$

Primjer 3.3.1 — ε-prijelazi u praksi

Kad želimo spojiti dva NKA N_1 i N_2 tako da prepoznaju konkatenaciju $L(N_1)L(N_2)$, to radimo tako da iz svakog završnog stanja od N_1 dodamo ε-prijelaz prema početnom stanju od N_2. Niti jedan znak se ne "gubi" — samo automati postaju lijepo spojeni.

3.4 Ekvivalentnost KA i NKA

Propozicija 3.4.1 (Sipser, teorem 1.39)**.** *Jezik je KA-regularan ako i samo ako je NKA-regularan.*

Smjer ($\Rightarrow$) je trivijalan: svaki KA je specijalan slucaj NKA (bez nedeterminizma i ε-prijelaza). Smjer ($\Leftarrow$) je zanimljiviji i koristi **partitivnu konstrukciju**.

3.4.1 Partitivna konstrukcija

Ideja: ako NKA u svakom trenutku može biti u više stanja odjednom, zasto ne bismo cijeli skup mogućih stanja proglasili jednim stanjem novog, determinističkog automata?

Definicija 3.4.1 — Partitivna konstrukcija

Neka je $N = (Q, \Sigma, \Delta, q_0, F)$ NKA. Definiramo KA

$$P(N) = \big(\mathcal{P}(Q),\ \Sigma,\ \delta,\ [\{q_0\}]_\varepsilon,\ \{T \subseteq Q : T \cap F \neq \emptyset\}\big),$$

gdje je

$$\delta(T, \alpha) := \big[\{q \in Q : (\exists p \in T)\ (p, \alpha, q) \in \Delta\}\big]_\varepsilon.$$

Intuicija: stanje $P(N)$-a je skup stanja u kojima N *može* biti. Kad pročitamo znak α, uzimamo sve Δ-nasljednike svakog stanja iz skupa, a zatim zatvorimo na ε-prijelaze.
Napomena. *Ako N ima n stanja, $P(N)$ ima 2^n stanja — eksponencijalan rast! U praksi se većinom koriste samo dostupna stanja (ona do kojih se uopće može doći iz početnog), kojih je obično znatno manje.*

Primjer 3.4.1 — Partitivna konstrukcija

Neka je N NKA s $Q = \{1, 2, 3\}$, $q_0 = 1$, $F = \{3\}$ i prijelazima:

$$\Delta = \{(1, a, 1),\ (1, a, 2),\ (2, b, 3),\ (3, b, 3)\}.$$

Ovaj NKA prepoznaje rijeci oblika $a^n b^k$ za $n \geq 1$, $k \geq 1$ (nejasno, ali pogledajmo partitivnu konstrukciju):
Početno stanje $P(N)$-a je $\{1\}$. Pratimo:

- $\delta(\{1\}, a) = \{1, 2\}$ (jer $(1, a, 1)$ i $(1, a, 2)$)
- $\delta(\{1, 2\}, a) = \{1, 2\}$ (iz 1 kao gore, iz 2 nema)
- $\delta(\{1, 2\}, b) = \{3\}$ (samo $(2, b, 3)$)
- $\delta(\{3\}, b) = \{3\}$ i $\delta(\{3\}, a) = \emptyset$

Dostupna stanja: $\{1\}$, $\{1, 2\}$, $\{3\}$, $\emptyset$ — samo 4 stanja umjesto $2^3 = 8$.

3.5 Zatvorenost na skupovne i jezicne operacije

Jedna od ključnih osobina regularnih jezika jest da su zatvoreni na mnoge prirodne operacije.

3.5.1 Skupovne operacije

Propozicija 3.5.1. *Klasa KA-regularnih jezika zatvorena je na* ***uniju***, ***presjek*** *i* ***komplement****.*

Dokaz za uniju i presjek koristi **Kartezijevu konstrukciju**: ako M_1 prepoznaje L_1 i M_2 prepoznaje L_2, konstruiramo KA koji "istovremeno" simulira oba:

$$M_1 \times M_2 := (Q_1 \times Q_2,\ \Sigma,\ \delta,\ (q_1, q_2),\ F),$$

gdje $\delta((q, q'), \alpha) := (\delta_1(q, \alpha), \delta_2(q', \alpha))$.

Za **uniju** stavimo $F = Q_1 \times F_2 \cup F_1 \times Q_2$. Za **presjek** stavimo $F = F_1 \times F_2$. Za **komplement** samo zamijenimo završna s nezavršnima: $F' = Q \setminus F$.

3.5.2 Jezicne operacije

Propozicija 3.5.2. *Klasa NKA-regularnih (= regularnih) jezika zatvorena je na* ***konkatenaciju***, ***Kleenevu zvijezdu*** *i* ***Kleenejev plus****.*

Ove konstrukcije su elegantnije s NKA:

- **Unija** $L \cup L'$: dodamo novo početno stanje s ε-prijelazima prema početnim stanjima oba automata.
- **Konkatenacija** LL': iz svakog završnog stanja od N dodamo ε-prijelaz prema početnom stanju od N'. Stara završna stanja od N više nisu završna.

- **Kleeneva zvijezda** L^*: dodamo novo početno završno stanje s ε-prijelazima prema starim početnim stanjima, te iz svakog završnog stanja ε-prijelaz natrag prema početnom stanju.

3.6 Kako dizajnirati KA: heuristike

Dizajn KA je vjestina koja se uci vjezbanjem. Nekoliko heuristika:

Ključna ideja

Pitajte se: sto KA mora pamtiti?
Stanja KA su različite "situacije" u kojima se automat može naći. Pitanje je: koje informacije o dosad pročitanom prefiksu su relevantne za odluku o prihvaćanju?
Primjeri:

- "Zadnji pročitani znak" — $|\Sigma|$ stanja.
- "Parnost broja a-ova" — 2 stanja.
- "Ostatak duljine modulo k" — k stanja.
- "Prefiks duljine do k" — konačno mnogo stanja.

Ako trebate pamtiti nešto *neograničeno* (npr. točni broj a-ova) — KA za to ne postoji.

Zadatak 3.6.1

(a) Konstruirajte KA nad $\{a, b\}$ koji prepoznaje jezik svih rijeci cija duljina je djeljiva s 3.

(b) Konstruirajte NKA koji prepoznaje jezik svih rijeci nad $\{0, 1\}$ koje završavaju s 010.

(c) Primijenite partitivnu konstrukciju na NKA iz (b) da dobijete ekvivalentni KA.

(d) Konstruirajte KA koji prepoznaje komplement jezika iz (a).

3.6 Sazetak

Konačni automat čita riječ znak po znak, ne može se vraćati unatrag, i pamti samo stanje u kojemu je. Usprkos ovim jakim ograničenjima, konačni automati prepoznaju točno regularnu klasu jezika.

$M = (Q, \Sigma, \delta, q_0, F)$	deterministički KA
$N = (Q, \Sigma, \Delta, q_0, F)$	nedeterministički NKA
$[S]_\varepsilon$	ε-ljuska skupa stanja
$P(N)$	partitivna konstrukcija (NKA $\to$ KA)
$M_1 \times M_2$	Kartezijeva konstrukcija (unija/presjek)

U sljedećem poglavlju zatvaramo krug: pokazujemo da su KA, NKA, regularni izrazi i linearne gramatike sve *isti* formalizam, samo u različitom ruhu.

Poglavlje 4

Ekvivalentnost klasa regularnih jezika

U dosad unijeli nekoliko nacina opisivanja regularnih jezika, a ovo poglavlje zatvara krug — pokazat cemo ekvivalentnost razlicitih formalizama.

4.1 Krug koji treba zatvoriti

Dosad smo uveli nekoliko nacina opisivanja regularnih jezika:

- **RI** — regularni izrazi (poglavlje 2)
- **KA** — deterministicki konacni automati (poglavlje 3)
- **NKA** — nedeterministicki konacni automati (poglavlje 3)
- **DLG** — desnolinearne gramatike (ovo poglavlje)
- **LLG** — lijevolinearne gramatike (ovo poglavlje)

Svaki od ovih formalizma definira neku klasu jezika. Kljucni teorem ovog poglavlja kaze da su sve te klase iste:

$$\mathrm{Reg}_{\mathrm{KA}} = \mathrm{Reg}_{\mathrm{NKA}} = \mathrm{Reg}_{\mathrm{RI}} = \mathrm{Reg}_{\mathrm{DLG}} = \mathrm{Reg}_{\mathrm{LLG}}.$$

Tu klasu jednostavno zovemo Reg — **klasa regularnih jezika**.

Dokaz ide "u krug": pokazujemo RI $\to$ NKA $\to$ KA $\to$ DLG $\to$ RI, a LLG ukljucujemo pomocu operacije reverza.

4.2 Desnolinearne gramatike (DLG)

Definicija 4.2.1 — Gramatika

Gramatika nad abecedom Σ je transformacijski sustav $G = (V, \Sigma, \to, S)$, gdje je:

- V — konacni skup **varijabli** (disjunktan od Σ),
- $Y := \Sigma \cup V$ — skup svih **simbola**,
- $S \in V$ — **pocetna varijabla**,
- $\to \subseteq Y^+ \times Y^*$ — konacna relacija **pravila** (na lijevoj strani barem jedna varijabla).

Pisemo $u \Rightarrow v$ ako postoji pravilo $s \to t$ i rijeci a, b takve da je $u = asb$ i $v = atb$. Gramatika **generira** jezik

$$L(G) := \{w \in \Sigma^* : S \Rightarrow^* w\}.$$

Ovo je opca definicija gramatike. Za regularne jezike dovoljne su nam gramatike s *jako* ogranicenim pravilima:

Definicija 4.2.2 — Desnolinearna gramatika

Gramatika G je **desnolinearna** (DLG) ako su sva njena pravila jednog od dva oblika:

$$A \to \alpha B \qquad \text{ili} \qquad A \to \varepsilon,$$

gdje su $A, B \in V$ varijable, a $\alpha \in \Sigma$ znak.

Intuicija: svako pravilo ili "pojede" jedan znak i zamijeni varijablu s novom varijablom (koja ce pojesti sljedeci), ili varijabla nestane (kraj izvoda). Varijabla je uvijek na *desnom* kraju medurjecita — otuda ime "desnolinearna".

Primjer 4.2.1

DLG s varijablama $\{S, A\}$ i pravilima $S \to aA$, $A \to aA$, $A \to bA$, $A \to \varepsilon$ generira jezik svih rijeci koje pocinju s a i zatim mogu imati bilo sto. Regularni izraz: $a(a \cup b)^*$.

4.3 Pretvorba KA → DLG

Svaki KA mozemo pretvoriti u ekvivalentnu DLG na prirodan nacin: stanja postaju varijable, prijelazi postaju pravila.

Propozicija 4.3.1. *Svaki KA-regularni jezik je DLG-regularan.*

Dokaz (konstrukcija). Neka je $M = (Q, \Sigma, \delta, q_0, F)$ KA koji prepoznaje jezik L. Konstruiramo DLG G ovako:

- Za svako stanje $q \in Q$ uvedemo varijablu V_q.
- Pocetna varijabla: $S := V_{q_0}$.
- Za svaki prijelaz $\delta(q, \alpha) = p$ dodamo pravilo $V_q \to \alpha V_p$.
- Za svako zavrsno stanje $q \in F$ dodamo pravilo $V_q \to \varepsilon$.

Moze se pokazati indukcijom da za svako stanje $q \in Q$ vrijedi

$$L(Q, \Sigma, \delta, q, F) = L(V, \Sigma, \to, V_q),$$

tj. varijabla V_q generira tocno rijeci koje KA prihvaca krenuvsi iz stanja q. Za $q := q_0$ dobijemo $L(M) = L(G)$. □

Primjer 4.3.1

KA za rijeci koje zavrsavaju na ab iz poglavlja 3 pretvara se u DLG:

$$\begin{aligned} V_{q_0} &\to aV_{q_a} \mid bV_{q_0} \\ V_{q_a} &\to aV_{q_a} \mid bV_{q_{ab}} \\ V_{q_{ab}} &\to aV_{q_a} \mid bV_{q_0} \mid \varepsilon \end{aligned}$$

Ovo generira tocno rijeci koje zavrsavaju na ab.

4.4 Gramaticki sustav i Ardenovo pravilo

Za smjer DLG → RI koristimo tehniku **gramatickih jednadžbi**.

4.4.1 Gramaticki sustav

Svaka DLG zadaje *sustav jednadžbi* nad jezicima: za svaku varijablu A skupimo sva njena pravila i zapisemo jednadžbu:

$$A = \alpha_1 B_1 \cup \alpha_2 B_2 \cup \cdots \cup \alpha_k B_k \ \cup \ (\varepsilon \text{ ako ima pravilo } A \to \varepsilon).$$

Ako A nema nijednog pravila, dodamo $A = \emptyset$.

Supstitucijom (uvrstavanjem jednadžbi jedne u drugu) pokusavamo eliminirati varijable i doci do regularnog izraza za S.

4.4.2 Problem: rekurzija

Supstitucija funkcionira kada nema rekurzije. Ali DLG cesto ima rekurzivna pravila poput $A = aA \cup b$, gdje se A pojavljuje s obje strane. Sto tada?

Propozicija 4.4.1 (Ardenovo pravilo)**.** *Jednadžba $X = AX \cup B$ ima rjesenje $X = A^*B$. Ako je A pozitivan, to je* jedino *rjesenje. Inace, to je* najmanji *skup koji zadovoljava jednadžbu.*

Provjera. Uvrstimo $X = A^*B$ u jednadžbu:

$$A(A^*B) \cup B = (AA^*)B \cup B = A^+B \cup \varepsilon B = (A^+ \cup \varepsilon)B = A^*B. \checkmark$$

Ardenovo pravilo je analogon formule za geometrijski red: $a \cdot r + b = r$ ima rjesenje $r = b/(1-a)$, tj. $r = a^*b$ u jezicnoj algebri.

Primjer 4.4.1 — Pretvorba DLG $\to$ RI

Pretvorimo DLG s pravilima:

$$S \to aA \mid bB, \quad A \to aA \mid bA \mid aN, \quad B \to aB \mid bB \mid bN, \quad N \to \varepsilon.$$

Gramaticki sustav ($N = \varepsilon$ odmah uvrstimo):

$$\begin{aligned} S &= aA \cup bB \\ A &= aA \cup bA \cup a = (a \cup b)A \cup a \end{aligned}$$

Po Ardenovom pravilu ($a \cup b$ je pozitivan): $A = (a \cup b)^*a$.
Analogno: $B = (a \cup b)^*b$.
Uvrstimo u S:

$$S = a(a \cup b)^*a \cup b(a \cup b)^*b.$$

To su sve rijeci koje pocinju i zavrsavaju istim slovom! ✓

4.5 RI $\to$ NKA: Thompsonova konstrukcija

Sad zatvaramo krug s druge strane: svaki regularni izraz mozemo pretvoriti u NKA. Dokaz ide indukcijom po izgradnji izraza.

Propozicija 4.5.1. *Svaki RI-regularni jezik je NKA-regularan.*

Dokaz (Thompsonova konstrukcija). Za svaki inicijalni jezik konstruiramo trivijalni NKA:

- $\emptyset$: jedno stanje, nema prijelaza, nema zavrsnih stanja.
- ε: jedno stanje, ono je i pocetno i zavrsno, nema prijelaza.
- $\{\alpha\}$: dva stanja, prijelaz α od pocetnog do zavrsnog.

Za slozenije izraze koristimo konstrukcije iz poglavlja 3 (unija s ε-prijelazima, konkatenacija, zvijezda). □

4.6 Lijevolinearne gramatike i reverzi

Definicija 4.6.1 — Lijevolinearna gramatika

Gramatika je **lijevolinearna** (LLG) ako su sva njena pravila oblika:

$$A \to B\alpha \qquad \text{ili} \qquad A \to \varepsilon,$$

dakle varijabla je na *lijevom* kraju desne strane pravila.

LLG generiraju iste jezike kao i DLG (a time i sve ostale klase), ali to nije odmah ocito. Dokazujemo to pomocu **reverza**.

Definicija 4.6.2 — Reverz

Reverz rijeci $w = \alpha_1\alpha_2\cdots\alpha_n$ je $w^R = \alpha_n\cdots\alpha_2\alpha_1$. **Reverz jezika** je $L^R = \{w^R : w \in L\}$.

Za regularne izraze vrijede jednostavne formule:

$$\emptyset^R = \emptyset, \quad \varepsilon^R = \varepsilon, \quad \alpha^R = \alpha,$$

$$(LM)^R = M^R L^R, \quad (L \cup M)^R = L^R \cup M^R, \quad (L^*)^R = (L^R)^*.$$

Iz ovih formula indukcijom slijedi:
Propozicija 4.6.1. *Reverz svakog RI-regularnog jezika je ponovo RI-regularan. Dakle, klasa* $\mathrm{Reg}_{\mathrm{RI}}$ *zatvorena je na reverz.*

Sada, DLG koja generira jezik L lako se pretvori u LLG koja generira L^R (samo okrenimo redoslijed u pravilima). Kako je $\mathrm{Reg}_{\mathrm{RI}}$ zatvorena na reverz, zakljucujemo:

$$\mathrm{Reg}_{\mathrm{LLG}} = (\mathrm{Reg}_{\mathrm{DLG}})^R = (\mathrm{Reg}_{\mathrm{RI}})^R = \mathrm{Reg}_{\mathrm{RI}}.$$

4.7 Zakljucak: sve je isto

Evo kompletnog kruga pretvorbi:

Od	Do	Metoda
RI	NKA	Thompsonova konstrukcija (indukcija po izrazu)
NKA	KA	Partitivna konstrukcija
KA	DLG	Stanja $\leftrightarrow$ varijable, prijelazi $\leftrightarrow$ pravila
DLG	RI	Gramaticki sustav + Ardenovo pravilo
DLG	LLG	Reverz (okretanje pravila)

Ključna ideja

Sve ove pretvorbe su algoritamske — dajte mi jedno, dat cu vam bilo koje drugo. Svi formalizmi su jednako mocni; razlika je samo u pogodnosti za razlicite svrhe. RI je dobar za opis, KA za implementaciju, DLG za vezu s gramatikama.

Zadatak 4.7.1

(a) Pretvorite regularni izraz $a^*(b \cup c)^+$ u NKA Thompsonovom konstrukcijom.

(b) Primijenite partitivnu konstrukciju na rezultat.

(c) Pretvorite KA koji ste dobili u DLG.

(d) Rijesite gramaticki sustav te DLG i dobijte regularni izraz. Usporedite s polaznim izrazom.

(e) Napisite LLG ekvivalentnu DLG iz (c).

4.7 Sazetak

Pokazali smo da pet naizgled razlicitih formalizma opisuju istu klasu:

$\mathrm{Reg}_{\mathrm{KA}}$	jezici KA
$\mathrm{Reg}_{\mathrm{NKA}}$	jezici NKA
$\mathrm{Reg}_{\mathrm{RI}}$	regularni izrazi
$\mathrm{Reg}_{\mathrm{DLG}}$	desnolinearne gramatike
$\mathrm{Reg}_{\mathrm{LLG}}$	lijevolinearne gramatike

Sve su iste klase. Tu klasu zovemo Reg — **regularni jezici**. U sljedecem poglavlju prelazimo na bezkontekstne jezike — prvu pravu nadklasu.

Poglavlje 5

Bezkontekstni jezici

Regularni jezici ne mogu prebrojati. Cim je potrebno "zapamtiti" proizvoljan broj i usporediti ga s necim drugim, regularnost otkazuje. Bezkontekstni jezici rjesavaju upravo taj problem — dodavanjem stoga. Ovo poglavlje uvodi bezkontekstne gramatike, potisne automate, dokazuje njihovu ekvivalentnost, i opisuje kljucni algoritam CYK.

5.1 Motivacija i definicija BKG

Prisjetimo se: jezik $L_= = \{a^n b^n \mid n \geq 1\}$ nije regularan jer KA ne moze istovremeno pratiti broj a-ova i usporedivati ga s brojem b-ova — konacno mnogo stanja nije dovoljno.

No postoji gramatika koja ga generira:

$$S \to aSb \mid ab.$$

Lako se vidi indukcijom da ova gramatika generira tocno $a^n b^n$: pravilo $S \to aSb$ dodaje po jedan a s lijeva i jedan b s desna, a $S \to ab$ zatvara rekurziju.

Ova gramatika je **bezkontekstna**: svako pravilo zamjenjuje jednu varijablu nizom simbola, bez obzira na kontekst oko nje.

Definicija 5.1.1 — Bezkontekstna gramatika

Bezkontekstna gramatika (BKG) je gramatika $G = (V, \Sigma, \to, S)$ u kojoj je svako pravilo oblika

$$A \to w, \qquad A \in V,\ w \in (\Sigma \cup V)^*.$$

Dakle, lijeva strana je uvijek tocno jedna varijabla. Jezik koji G generira je $L(G) = \{w \in \Sigma^* : S \Rightarrow^* w\}$. Jezik je **bezkontekstan** (BK) ako ga generira neka BKG.

Napomena. *Svaka DLG je BKG (pravila $A \to \alpha B$ i $A \to \varepsilon$ ocito su bezkontekstna). Dakle,* Reg $\subseteq$ BK. *Jezik $L_=$ pokazuje da je inkluzija prava:* Reg $\subsetneq$ BK.

Primjer 5.1.1 — Aritmeticki izrazi

BKG za jednostavne aritmeticke izraze nad varijablom x i brojevima:

$$\begin{aligned} E &\to E + T \mid T \\ T &\to T * F \mid F \\ F &\to (E) \mid x \mid 0 \mid 1 \mid \dots \end{aligned}$$

Ova gramatika odrazava prioritet operatora: mnozenje (T) ima visi prioritet od zbrajanja (E), a zagrade (F) imaju najvisi. Iz E mozemo izvesti npr. $x + x * x$, i stablo parsiranja automatski reflektira ispravnu strukturu $(x + (x * x))$.

Primjer 5.1.2 — Zadatak sa slajdova

Koji jezik generira gramatika:

$$S \to 0J \mid 1N \mid \varepsilon, \quad J \to 1S \mid SJ, \quad N \to 0S \mid SN\,?$$

Svaki put kad iz S odemo u $0J$, sljedeci znak mora biti 1 (jer $J \to 1S$), sto znaci svaki 0 ima par 1 iza sebe. Slicno, svaki 1 ima par 0 iza sebe. Ovo generira jezik svih binarnih rijeci s jednakim brojem 0-ova i 1-ova — i taj je jezik bezkontekstan, ali nije regularan.

5.2 Zatvorenost klase BK

Propozicija 5.2.1. *Klasa* BK *zatvorena je na uniju, konkatenaciju i Kleeneve operacije.*

Dokaz (skica). Za BKG G_1 i G_2 s disjunktnim skupovima varijabli koje generiraju L_1 i L_2:

- **Unija**: dodamo novu pocetnu varijablu S i pravilo $S \to S_1 \mid S_2$.
- **Konkatenacija**: dodamo S i pravilo $S \to S_1 S_2$.
- **Zvijezda**: dodamo S i pravilo $S \to S_1 S \mid \varepsilon$.

Ključna ideja

Za razliku od regularnih jezika, klasa BK **nije** zatvorena na presjek ni na komplement. Ovo je bitna razlika: Reg je zatvorena na sve skupovne operacije, a BK nije. Dokaz dolazi u poglavlju 6 (lema o pumpanju za BK).
Ipak vrijedi **konusno svojstvo**: presjek bezkontekstnog i regularnog jezika je bezkontekstan.

5.3 Potisni automati (PA)

KA je premoczan za regularne, a premoczan — ne, preslabacak — za bezkontekstne jezike. Treba mu memorija. Najjednostavniji dodataak: **stog** (stack, magacin).

Definicija 5.3.1 — Potisni automat

Potisni automat (PA) je uredena sestorka

$$M = (Q, \Sigma, \Gamma, \Delta, q_0, F),$$

gdje su Q, Σ, q_0, F kao kod NKA, a novo je:

- Γ — **abeceda stoga** (moze se razlikovati od Σ),
- $\Delta \subseteq Q \times \Sigma_\varepsilon \times \Gamma_\varepsilon \times Q \times \Gamma_\varepsilon$ — relacija prijelaza.

Prijelaz $(q, \alpha, t, p, s) \in \Delta$ znaci: "u stanju q, citajuci α (ili ε), s t na vrhu stoga (ili ε ako stog ne gledamo), predi u stanje p i zamijeni t s s na stogu."

Prijelaz moze kombinirati razlicite akcije:

t	s	Sto se dogada
ε	γ	**push**: stavi γ na stog
γ	ε	**pop**: skini γ sa stoga
γ	γ	provjeri vrh stoga (ostavi nepromijenjenim)
ε	ε	ignoriraj stog

Napomena. *Za PA je **nedeterminizam kljucan**. Deterministicki PA prepoznaju uzu klasu od svih BK-jezika (postoje BK-jezici koje deterministicki PA ne moze prepoznati). Ovo je bitna razlika od KA, gdje determinizam i nedeterminizam daju istu klasu.*

5.3.1 Jednostavni PA (JPA)

Radi lakse teorije, definiramo **jednostavni PA** (JPA) koji ima tri dodatna svojstva:

1. Ima tocno jedno zavrsno stanje q_X.
2. Kad dosegne q_X, stog mu je prazan.
3. Svaki prijelaz je iskljucivo *push* ili *pop* (ne oboje).

Svaki PA moze se pretvoriti u ekvivalentni JPA: uvedemo novi simbol \$ na dno stoga, sva bivsa zavrsna stanja isprazne stog i odu u q_X, a svaki "kombinirani" prijelaz rastavimo na dva.

Primjer 5.3.1 — PA za $L_=$

Konstruirajmo PA koji prepoznaje $L_= = \{a^n b^n \mid n \geq 1\}$.
Ideja: stavljamo a-ove na stog dok citamo a-ove, pa skidamo po jedan za svaki b

koji procitamo.

- Stanja: $\{q_0, q_1, q_{\text{acc}}\}$, pocetno q_0, zavrsno $\{q_{\text{acc}}\}$.
- Na pocetku push \$ (marker dna stoga).
- U q_0: za svaki a, push a (ostani u q_0).
- Prijelaz u q_1: ε-prijelaz (pocetak b-ova).
- U q_1: za svaki b, pop a.
- Kad je na vrhu stoga \$, prijelaz u q_{acc}.

5.4 Ekvivalentnost BKG i PA

Propozicija 5.4.1 (Sipser, lema 2.21 i teorem 2.20)**.** *Jezik je bezkontekstan ako i samo ako ga prepoznaje neki PA.*

5.4.1 BKG → PA

Konstruiramo PA koji "pogada" izvod rijeci iz BKG:

1. Push \$ i pocetnu varijablu S na stog.
2. Ponavljaj:
   - Ako je na vrhu stoga varijabla A: nedeterministicno odaberi pravilo $A \to w$ i zamijeni A sa simbolima od w (u obrnutom redoslijedu, da prvi simbol bude na vrhu).
   - Ako je na vrhu stoga znak $\alpha \in \Sigma$: procitaj sljedeci ulazni znak; ako se slaze, pop; inace odbaci granu.
   - Ako je na vrhu stoga \$: prihvati.

Kljucna napomena: ovaj PA koristi nedeterminizam da "pogodi" tocni lijevi izvod. PA prolazi tocno kroz konfiguracije koje odgovaraju koracima lijevog izvoda.

5.4.2 PA → BKG

Ovaj smjer je tehnicki zahtjevniji. Svaki PA pretvorimo u JPA, a zatim za svaki par stanja (p, q) uvedemo varijablu ${}_pV_q$ sa znacenjem: "JPA iz stanja p s praznim stogom moze procitati neku rijec i doci u stanje q s praznim stogom".

Pravila gramatike odrazavaju dva slucaja:

- **Stog postane prazan u meduvremenu**: postoji stanje r gdje stog postane prazan, pa se izvod "lomi" na dva dijela:

$$ {}_pV_q \to {}_pV_r \; {}_rV_q. $$

- **Stog nikad nije prazan**: isti simbol γ koji je pocetak push na kraju pop-a. Ako je push na pocetku $(p, \alpha, \varepsilon, r, \gamma)$ i pop na kraju $(s, \beta, \gamma, q, \varepsilon)$:

$$ {}_pV_q \to \alpha \; {}_rV_s \; \beta. $$

Pocetna varijabla je ${}_{q_0}V_{q_X}$.

5.5 Chomskyjeva normalna forma (ChNF)

Svaka BKG moze se pretvoriti u kanonski oblik koji je koristan za algoritme i dokaze.

Definicija 5.5.1 — Chomskyjeva normalna forma

BKG je u **Chomskyjevoj normalnoj formi** (ChNF) ako su sva njena pravila jednog od dva oblika:

$$A \to BC \qquad \text{(sirenje)}, \qquad A \to \alpha \qquad \text{(citanje)},$$

gdje su $A, B, C \in V$ varijable (s uvjetom $S \notin \{B, C\}$), a $\alpha \in \Sigma$ znak.

Svaka ChNF gramatika generira pozitivan jezik (jer ni sirenje ni citanje ne mogu dati ε). Dokazat cemo i obrat: svaki pozitivni BK-jezik ima ChNF gramatiku.

5.5.1 Algoritam pretvorbe u ChNF (5 faza)

Pretvorba ide u pet faza, svaka osigurava jedno svojstvo ne kvareci prethodna.

Faza START. Ako se pocetna varijabla S pojavljuje na desnoj strani ijednog pravila, uvedemo novu varijablu S_0 s pravilom $S_0 \to S$ i proglasimo S_0 novom pocetnom varijablom.

Faza TERM. Za svaki znak $\alpha \in \Sigma$ koji se pojavljuje u nekom pravilu uz jos nesto (dakle nije sam), uvedemo novu varijablu N_α s pravilom $N_\alpha \to \alpha$, i sve takve pojave α zamijenimo s N_α.

Primjer: pravilo $A \to aBc$ postaje $A \to N_a B N_c$ (gdje su $N_a \to a$ i $N_c \to c$ nova pravila citanja).

Faza BIN. Za svako pravilo s $n \geq 3$ simbola na desnoj strani, $A \to V_1 V_2 \cdots V_n$, uvedemo nove varijable $A_1, \ldots, A_{n-2}$ i rastavimo ga na lanac:

$$A \to V_1 A_1, \quad A_1 \to V_2 A_2, \quad \ldots, \quad A_{n-2} \to V_{n-1} V_n.$$

Faza DEL. Eliminiramo pravila oblika $A \to \varepsilon$ (za $A \neq S$). Algoritam:

1. Nademo sve **nepozitivne varijable** $\mathrm{np} = \{A : A \Rightarrow^* \varepsilon\}$ iteracijom: pocnemo s $\mathrm{np} = \emptyset$, i dodajemo A kad su sve varijable na desnoj strani nekog pravila za A u np.
2. Za svako pravilo koje sadrzi $k > 0$ varijabli iz np, dodamo 2^k novih pravila (jedno za svaki podskup tih pojava koje brisemo), i brisemo originalno pravilo $A \to \varepsilon$.

Gramatika je bila pozitivna ako i samo ako $S \notin \text{np}$.

Faza UNIT. Eliminiramo pravila oblika $A \to B$ (jedna varijabla na desnoj strani). Za svako takvo pravilo: uklonimo ga, zapamtimo, i za svako pravilo $B \to u$ dodamo $A \to u$ (osim ako smo to pravilo vec uklonili).

Primjer 5.5.1 — Pretvorba u ChNF

Pretvorimo gramatiku $S \to ASA \mid aB$, $A \to B \mid S$, $B \to b \mid \varepsilon$.
START: S se ne pojavljuje na desnoj strani — preskacemo.
TERM: a i b se pojavljuju uz ostalo: uvedemo $N_a \to a$, $N_b \to b$. Dobijemo: $S \to ASA \mid N_aB$, $B \to N_b \mid \varepsilon$.
BIN: $S \to ASA$ ima tri simbola: $S \to AA_1$, $A_1 \to SA$.
DEL: $B \to \varepsilon$, pa $\text{np} = \{B\}$. Pravilo $S \to N_aB$ daje i $S \to N_a$.
UNIT: $A \to B$ i $A \to S$. Za $A \to B$: B ima pravila $B \to N_b$, pa dodamo $A \to N_b$. Za $A \to S$: S ima pravila $S \to AA_1 \mid N_aB \mid N_a$, pa dodamo ta pravila za A.

5.6 Algoritam CYK

ChNF nam omogucuje efikasan algoritam parsiranja.
Propozicija 5.6.1. *Problem prihvacanja A_{BK} je odluciv. Konkretno, algoritam CYK rjesava ga u $O(n^3 \cdot |G|)$ vremenu, gdje je $n = |w|$ duljina ulazne rijeci.*

5.6.1 Ideja: dinamicko programiranje

Kljucna opservacija za ChNF: ako $A \Rightarrow^* w_{i:j}$ (podrijec od pozicije i do j), tada prvi korak mora biti sirenje $A \to BC$, a B i C izvode neprazne podrijeci. Dakle postoji $k \in (i, j)$ takav da $B \Rightarrow^* w_{i:k}$ i $C \Rightarrow^* w_{k:j}$.

Ovo sugerira **dinamicko programiranje**: rjesavamo poprobleme $A \,?\, w_{i:j}$ redom od kracih prema duljima.

5.6.2 Tablica i algoritam

Tablicu T dimenzija $|V| \times n \times n$ popunjavamo:

Baza ($j = i + 1$, jednoznakovna podrijec):

$$T[A][i][i+1] = \text{True} \iff (A \to w_i) \in G.$$

Korak ($j - i = \ell > 1$):

$$T[A][i][j] = \text{True} \iff \exists\, (A \to BC) \in G,\ k \in (i, j) : T[B][i][k] \wedge T[C][k][j].$$

Odgovor: $w \in L(G) \iff T[S][0][n] = \text{True}$.

Primjer 5.6.1 — CYK na djelu

Neka je gramatika (u ChNF): $S \to AB \mid BC$, $A \to BA \mid a$, $B \to CC \mid b$, $C \to AB \mid a$ i rijec $w = baaba$.
Duljina je 5, tablica je 5×5. Popunjavamo redak po redak (duljine 1, 2, 3, 4, 5):

	b	a	a	b	a
	$w_{0:1}$	$w_{1:2}$	$w_{2:3}$	$w_{3:4}$	$w_{4:5}$
S		$\{S\}$ ok?	...	...	
A		A	A, C		A, C
B	B			B	
C		A, C	A, C		A, C

(Za punu tablicu pogledajte Sipser poglavlje 2.3 ili https://youtu.be/EloKZv84hUc?t=246.)
Na kraju provjerimo $T[S][0][5]$ — ako je True, rijec je u jeziku.

5.6.3 Rekonstrukcija stabla parsiranja

Osim bool vrijednosti, u tablicu mozemo zapisati i "svjedoke": trojke (B, C, k) koje su dovele do True. Tada mozemo rekonstruirati cijelo stablo parsiranja rekurzivnim obilaskom tablice.

Ključna ideja

CYK je kubne *vremenske* i kvadratne *prostorne* sloznosti za jednoznacne gramatike. Nedostatak: radi na ChNF, ne na originalnoj gramatici, pa stablo parsiranja sadrzi "artifakte" pretvorbe u ChNF.

5.7 Stabla parsiranja i viseznacanost

Definicija 5.7.1 — Stablo parsiranja

Stablo parsiranja iz BKG G za rijec w je stablo u cijem korijenu je pocetna varijabla, svaki unutarnji cvor je varijabla cija djeca odgovaraju desnoj strani nekog pravila, a listovi cine rijec w (citani slijeva nadesno).

Vrijedi: $w \in L(G)$ ako i samo ako postoji stablo parsiranja za w.

Definicija 5.7.2 — Viseznacanost

Rijec w je G-**viseznacna** ako postoji vise razlicitih stabala parsiranja za w u G. Gramatika je **viseznacna** ako postoji viseznacna rijec. Jezik je **viseznacan** ako je svaka njegova BKG viseznacna.

Primjer 5.7.1

Gramatika $S \to SS \mid a$ je viseznacna: rijec aaa ima dva razlicita stabla parsiranja ($(aa)a$ i $a(aa)$).
Pazi: vise izvoda ne znaci nuzno viseznacnost! Bitna su stabla parsiranja, ne izvodi. Lijevi izvod je jedinstven po stablu parsiranja — postoji bijekcija lijevi izvodi $\leftrightarrow$ stabla parsiranja.

5.7 Sazetak

BKG	bezkontekstna gramatika: $A \to w$
PA	potisni automat s stogom
JPA	jednostavni PA (jedan ulaz, push/pop odvojeni)
ChNF	$A \to BC$ ili $A \to \alpha$; korisna za algoritme
CYK	$O(n^3)$ algoritam parsiranja pomoc dinamickog programiranja
Viseznacanost	vise stabala parsiranja za istu rijec

U sljedecem poglavlju dokazujemo da su neke jezike nemoguce opisati BKG-om — pomocu **leme o pumpanju za BK**.

Poglavlje 6

Leme o pumpanju

Ta stvar zove se ***svojstvo pumpanja****.*

Intuicija kaze "treba pamtiti neograniceno puno", ali to nije formalni dokaz. Leme o pumpanju daju nam precizno oruzje: ako jezik nema odredeno "periodicno" svojstvo, onda ne moze biti u toj klasi.

6.1 Lema o pumpanju za regularne jezike

Propozicija 6.1.1 (Lema o pumpanju, Sipser 1.70)**.** *Neka je L regularan jezik. Tada postoji broj $p \geq 1$ ("duljina pumpanja") takav da se svaka rijec $w \in L$ duljine $|w| \geq p$ moze zapisati kao $w = xyz$ uz uvjete:*

(i) $y \neq \varepsilon$ *(pumpa je neprazna),*

(ii) $|xy| \leq p$ *(pumpa je blizu pocetka),*

(iii) $xy^iz \in L$ *za sve* $i \geq 0$ *(pumpanje cuva jezik).*

Dokaz. Neka KA M prepoznaje L, i neka je $p = |Q|$ broj stanja. Za rijec $w \in L$ s $|w| \geq p$: citajuci prvih p znakova, automat prolazi kroz $p+1$ stanja $r_0, r_1, \ldots, r_p$. Po **Dirichletovom principu** (golubinjak), neka dva stanja su ista: $r_i = r_j$ za neke $0 \leq i < j \leq p$.

Definirajmo:

$$x = w_{0:i}, \quad y = w_{i:j}, \quad z = w_{j:|w|}.$$

Tada:

- $y \neq \varepsilon$ jer $j > i$, dakle $|y| = j - i > 0$.
- $|xy| = j \leq p$.
- Automat koji procita xy^iz: ulazi u r_i nakon x, petlja i puta kroz $r_i \to \cdots \to r_j = r_i$, zatim nastavlja od $r_j = r_i$ kroz z i zavrsava u istom zavrsnom stanju kao i za w. $\square$

Ključna ideja

Lemu koristimo **kontrapositivno**: ako za svaki p mozemo naci rijec $w \in L$ koja se ne moze pumpati (za svaki rastav xyz koji zadovoljava (i) i (ii), postoji i takav da $xy^iz \notin L$), tada L nije regularan.
Dokaz je **igra**: nas protivnik odabire p, mi biramo w, protivnik bira rastav xyz, mi biramo i. Pobjedujemo ako nasmo $xy^iz \notin L$.

6.2 Primjena: $L_=$ nije regularan

Propozicija 6.2.1. *Jezik $L_= = \{a^nb^n \mid n \geq 1\}$ nije regularan.*

Dokaz. Pretpostavimo suprotno: neka je $L_=$ regularan s duljinom pumpanja p.

Odaberemo rijec $w = a^pb^p \in L_=$ (duljine $2p \geq p$, dakle zadovoljava uvjet). Prema lemi, postoji rastav $w = xyz$ s $y \neq \varepsilon$ i $|xy| \leq p$.

Iz $|xy| \leq p = |a^p|$ slijedi da x i y leze unutar bloka a-ova, dakle $y = a^m$ za neki $m \geq 1$.

Pumpajmo s $i = 0$: $xy^0z = xz = a^{p-m}b^p$. Da bi ovo bilo u $L_=$, trebamo jednak broj a-ova i b-ova: $p - m = p$, ali $m \geq 1$ pa $p - m < p$. Kontradikcija: $xz \notin L_=$. □

Primjer 6.2.1 — Tipicna strategija dokaza

Struktura dokaza je uvijek ista:

1. **Pretpostavimo** da je L regularan s duljinom pumpanja p.
2. **Odaberemo** konkretnu rijec $w \in L$ duljine $\geq p$ (tipicno w ovisi o p, npr. $w = a^pb^p$).
3. **Analiziramo** sve moguce rastave xyz koji zadovoljavaju uvjete (i) i (ii). Cesto uvjet $|xy| \leq p$ snazno ogranicava gdje moze lezati y.
4. **Pumpamo** s nekim i (tipicno $i = 0$ ili $i = 2$) i pokazujemo da $xy^iz \notin L$.
5. **Kontradikcija**: L nije regularan.

Primjer 6.2.2 — $\{w \in \{a,b\}^* : |w|_a = |w|_b\}$ nije regularan

Neka je $L = \{w : |w|_a = |w|_b\}$. Odaberemo $w = a^pb^p$. Sve kao gore: $y = a^m$, pumpanje s $i = 0$ daje $a^{p-m}b^p$ s razlicitim brojem a-ova i b-ova. □

Primjer 6.2.3 — $\{0^n : n$ je prost$\}$ nije regularan

Neka je $L = \{0^n : n \text{ je prost}\}$. Odaberemo $w = 0^p$ gdje je p prvi prost broj $\geq p$ duljine pumpanja (takav postoji jer ima beskonacno prostih).
Rastav: $w = xyz$, $|y| = m \geq 1$, $|xy| \leq p$. Pumpamo s $i = |xz| + 1$: duljina od xy^iz je $|x| + i \cdot m + |z| = |xz| + im = |xz|(1 + m) + m \cdot |xz| \ldots$
Koristimo drugi trik: $|xy^iz| = |xz| + im$. Uzmemo $i = |xz|$, tada $|xy^iz| = |xz| + |xz| \cdot m = |xz|(1+m)$ — djeljivo s $|xz|$ i s $(1+m)$, dakle nije prost za $|xz| > 1$. □

Zadatak 6.2.1

Dokazite da sljedeci jezici nisu regularni (koristeci lemu o pumpanju):
(a) $\{a^n b^{2n} \mid n \geq 0\}$
(b) $\{w \in \{a,b\}^* : w = w^R\}$ (palindromi)
(c) $\{1^{n^2} \mid n \geq 0\}$ (kvadrati u unarnom zapisu)
(d) $\{a^i b^j c^k \mid i + j = k\}$

6.3 Lema o pumpanju za bezkontekstne jezike

Ista ideja, ali za BK-jezike i s pet dijelova umjesto tri.

Propozicija 6.3.1 (Lema o pumpanju za BK, Sipser 2.34)**.** *Neka je L bezkontekstan jezik. Tada postoji broj $p \geq 1$ takav da se svaka rijec $s \in L$ s $|s| \geq p$ moze zapisati kao $s = uvxyz$ uz uvjete:*

(i) $vy \neq \varepsilon$ *(barem jedna od pumpi je neprazna),*

(ii) $|vxy| \leq p$ *(pumpe su blizu jedna drugoj),*

(iii) $uv^i xy^i z \in L$ *za sve* $i \geq 0$ *(pumpanje cuva jezik).*

Ideja dokaza. Neka BKG G (u ChNF) generira L, i neka je $b = \max |desnastrana|$ (za ChNF, $b = 2$). Za dovoljno dugu rijec s, stablo parsiranja mora imati duboku granu — toliko duboku da se neka varijabla R mora ponoviti duz iste grane (po Dirichletovom principu, ako grana ima vise od $|V|$ cvorova).

Istaknemo zadnja dva pojavljivanja R u toj grani:

$$S \Rightarrow^* uRz \Rightarrow^* uvRyz \Rightarrow^* uvxyz = s.$$

Budici da $R \Rightarrow^* vRy$, mozemo iterirati: $R \Rightarrow^* v^i R y^i$, a zatim zatvoriti s $R \Rightarrow^* x$. Time dobijemo $uv^i xy^i z \in L$ za svaki $i \geq 0$.

Uvjet $|vxy| \leq p$ dolazi iz toga da biramo *zadnja* dva pojavljivanja R u dubini: podstablo ispod nizeg R nije preduboko, pa rijec koju ono generira nije preduga. □

6.4 Primjena: $L_{\equiv}$ nije bezkontekstan

Propozicija 6.4.1. *Jezik $L_{\equiv} = \{a^n b^n c^n \mid n \geq 1\}$ nije bezkontekstan.*

Dokaz. Pretpostavimo suprotno: neka je p duljina pumpanja za $L_{\equiv}$.

Odaberemo $s = a^p b^p c^p$ (duljine $3p \geq p$). Prema lemi, postoji rastav $s = uvxyz$ s $vy \neq \varepsilon$ i $|vxy| \leq p$.

Kljucna opservacija: $|vxy| \leq p$, a a-ovi i c-ovi su udaljeni $2p > p$ znakova. Dakle vxy ne moze sadrzavati i a-ove i c-ove istovremeno.

Slucaj 1: vxy ne sadrzi c-ove. Tada je svih p slova c u dijelu z. Pumpamo s $i = 2$: rijec uv^2xy^2z ima vise a-ova ili b-ova (jer smo pumpali v ili y koji sadrze a-ove ili b-ove), ali i dalje p slova c. Dakle $uv^2xy^2z \notin L_{\equiv}$.

Slucaj 2: vxy ne sadrzi a-ove (analogno, ili BNSO). Svih p slova a je u u. Pumpamo s $i = 0$: uxz ima p slova a, ali manje od p slova b ili c (jer $vy \neq \varepsilon$ pa smo nesto uklonili). Dakle $uxz \notin L_{\equiv}$. □

Primjer 6.4.1 — Nezatvorenost BK na presjek

Definiramo:

$$L_{a=b} = \{a^n b^n c^k : n, k \geq 1\} = L_{=} \cdot c^+,$$
$$L_{b=c} = \{a^k b^n c^n : n, k \geq 1\} = a^+ \cdot L_{=}.$$

Oba su bezkontekstna (konkatenacija BK i regularnog). Ali $L_{a=b} \cap L_{b=c} = L_{\equiv}$ — a upravo smo dokazali da $L_{\equiv}$ nije bezkontekstan!
Zakljucak: **BK nije zatvorena na presjek**. Iz de Morgana slijedi ni na komplement.

Zadatak 6.4.1

Dokazite da sljedeci jezici nisu bezkontekstni:
(a) $\{a^n b^n c^n d^n \mid n \geq 1\}$
(b) $\{ww \mid w \in \{a, b\}^*\}$ (kvadrati)
(c) $\{a^i b^j c^k \mid i < j < k\}$
(d) $\{1^{n^2} \mid n \geq 0\}$ (je li ovo BK?)

6.5 Usporedba dviju lema

	Reg (lema za Reg)	**BK (lema za BK)**
Rastav	xyz (3 dijela)	$uvxyz$ (5 dijelova)
Pumpe	y (jedna pumpa)	v i y (dvije pumpe)
Uvjet nepraznosti	$y \neq \varepsilon$	$vy \neq \varepsilon$
Uvjet lokalizacije	$\lvert xy \rvert \leq p$	$\lvert vxy \rvert \leq p$
Pumpanje	$xy^iz \in L$	$uv^ixy^iz \in L$
Kljucni alat dokaza	Dirichlet na stanjima KA	Dirichlet na varijablama u stablu

Obje leme imaju isti oblik dokaza neregularnosti/nebezkontekstnosti: pretpostavi, odaberi rijec, analiziraj rastav, pumpa, kontradikcija.

Ključna ideja

Leme o pumpanju su **nuzni** uvjeti za regularnost/bezkontekstnost, ali ne i **dovoljni**! Postoje jezici koji zadovoljavaju lemu o pumpanju za Reg, a nisu regularni. Lema se koristi samo za dokazivanje *neregularnosti* — ne moze dokazati regularnost.

6.5 Sazetak

Lema za Reg	$w = xyz$, $y \neq \varepsilon$, $\lvert xy \rvert \leq p$, $xy^iz \in L$
Lema za BK	$s = uvxyz$, $vy \neq \varepsilon$, $\lvert vxy \rvert \leq p$, $uv^ixy^iz \in L$
Strategija	pretpostavi $\rightarrow$ odaberi rijec $\rightarrow$ analiziraj $\rightarrow$ pumpa $\rightarrow$ kontradikcija

U sljedecem poglavlju prelazimo na kontekstne jezike i ogranicene automate — prvu klasu za kojom trebamo "pravu" traku.

Poglavlje 7

Kontekstni jezici

Bezkontekstni jezici ne mogu prebrojati tri stvari odjednom. Kontekstni jezici rjesavaju i taj problem — dodavanjem trake ogranicene na duljinu ulaza. Klasa Kon je u mnogo cemu lijepa: zatvorena je na sve skupovne operacije, a prepoznavanje je odlucivo.

7.1 Motivacija: sto ne moze BK?

U poglavlju 6 dokazali smo da $L_{\equiv} = \{a^n b^n c^n \mid n \geq 1\}$ nije bezkontekstan. Intutivno: potisni automat moze uskladiti broj a-ova i b-ova (stavi a-ove na stog, skidaj za svaki b), ali jednom kad skine sve s hrpe, vise ne zna koliko je b-ova prebrojio — a to tocno treba za provjeru c-ova.

Rjesenje: umjesto stoga koji nestaje, trebamo traku po kojoj se mozemo kretati u oba smjera. Kljucno ogranicenje: traka je dugacka *tocno koliko i ulazna rijec*. Nema vise memorije od ulaza.

7.2 Ograniceni automat (OA)

Definicija 7.2.1 — Ograniceni automat

Ograniceni automat (OA) je uredena sestorka

$$O = (Q,\ \Sigma,\ \Gamma,\ \Delta,\ q_0,\ q_X),$$

gdje su Q, $\Sigma \subseteq \Gamma$, q_0, q_X kao obicno, a

$$\Delta \subseteq Q \times \Gamma \times Q \times \Gamma \times \{-1, 1\}$$

je relacija prijelaza. Prijelaz (q, α, p, β, d) znaci: "u stanju q, citajuci α na poziciji m, predi u stanje p, zapisi β na poziciju m, te se pomakni na poziciju $m + d$ (ali ne izlazi s trake: $0 \leq m + d < |w|$)."

OA prihvaca rijec w ako iz pocetne konfiguracije $(q_0, 0, w)$ moze doci do neke zavrsne konfiguracije (q_X, m, t).

Napomena. *Kljucne razlike od PA:*

- *Traka je ogranicena na $|w|$ znakova — nema vise.*
- *Moze se kretati lijevo i desno (PA samo naprijed).*
- *Moze pisati po traci (modifikacija ulaza).*
- *Kao i PA: OA je* ***nedeterministican****, a deterministicni i nedeterministicni OA* nisu *ekvivalentni (za razliku od KA) — ovo je otvoreni problem!*

Primjer 7.2.1 — OA za $L_{\equiv}$

Pseudokod OA koji prepoznaje $L_{\equiv} = \{a^n b^n c^n\}$:

1. Oznaci prvi a i procitaj sva a dalje. Ako nema a, odbij.
2. Oznaci prvi b iza svih a-ova. Ako nema b, odbij.
3. Oznaci prvi c iza svih b-ova. Ako nema c, odbij.
4. Vrati se lijevo do zadnjeg oznacenog a.
5. Ponovi od koraka 1 s prvim neoznacenim a.
6. Kad su sva a oznacena: provjeri da su sva b i c takoder oznacena (ne smije biti neoznacenih). Prihvati.

Svaki prolaz oznaci tocno jedno a, jedno b i jedno c, sto osigurava $|a| = |b| = |c|$.

7.2.1 Odlucivost A_{OA}

Za razliku od Turingovih strojeva, za OA uvijek postoji konacno mnogo konfiguracija (duljina je fiksna $|w|$), pa mozemo problem prihvacanja rjesiti obilazak grafa:

Za OA O i rijec w, konstruiramo konacni graf ciji su vrhovi sve moguce konfiguracije duljine $|w|$:

$$\text{broj vrhova} = |Q| \cdot |w| \cdot |\Gamma|^{|w|}.$$

Problem prihvacanja svodi se na dostupnost zavrsnih konfiguracija iz pocetne — sto rjesavamo DFS-om ili BFS-om.

Napomena. *Graf je ogroman (eksponencijalan u $|w|$), pa algoritam nije polinoman. Ali je konacni, pa je problem* odluciv.

7.3 Poboljsanja OA

Osnovna definicija OA je prakticno nezgrapna. Pokazujemo niz ekvivalentnih poboljsanja:

Poboljsanje	Ideja simulacije
Nemijenjanje trake	svaki $(q, *, p, *, d)$ postaje $\lvert\Gamma\rvert$ prijelaza
Vise tragova	Kartezijev produkt abeceda tragova
Detekcija rubova	trag s oznakama $\{\wedge, \sqcup\}$
Stajanje na mjestu ($d = 0$)	lijevo pa desno, uz provjeru ruba
Vise zavrsnih stanja	novo stanje q_X, ε-prijelazi iz svakog $q \in F$
Konacne varijable	produkt $Q \times S$ za varijablu iz S
Vise trakova (s gl. nezavisnim)	$2k$ tragova, svaka glava kao oznaka

Svako od ovih poboljsanja ekvivalentno je originalnom OA u smislu da prepoznaje istu klasu jezika. Dokazi idu metodom **simulacije**: poboljsani stroj uvijek moze sve sto moze obicni OA (trivijalni smjer), a obicni OA simulira poboljsanog imitirajuci svaki elementarni korak.

7.4 Kontekstne i monotone gramatike

Definicija 7.4.1 — Kontekstna gramatika

Pravilo $uAv \to ubv$ je **kontekstno** ako su $u, v \in Y^*$, $A \in V$ i $b \in Y^+$. Dakle: varijabla A moze se zamijeniti s b samo u "kontekstu" $u \cdot v$ oko nje.
Kontekstna gramatika (KG) je gramatika u kojoj su sva pravila kontekstna. Jezik je **kontekstan** ako ga generira neka KG.

Definicija 7.4.2 — Monotona gramatika

Pravilo $u \to v$ je **monotono** ako $|u| \leq |v|$ — desna strana nije kraca od lijeve. Gramatika je **monotona** (MG) ako su joj sva pravila monotona.

Propozicija 7.4.1. *Klase* KG, MG *i* OA *generiraju/prepoznaju istu klasu jezika:* ***kontekstne jezike*** *Kon. Vrijedi* $\mathrm{BK}_+ \subsetneq \mathrm{Kon}$ *(svi pozitivni BK-jezici su kontekstni, ali* $L_\equiv$ *je kontekstan a nije BK).*

7.4.1 Monotona gramatika za $L_\equiv$

Primjer monotone gramatike koja generira $L_\equiv$:

$$S \to abc \mid aSBc, \quad cB \to Bc, \quad bB \to bb.$$

Izvod $a^n b^n c^n$ koristi pravilo $S \to aSBc$ tocno $n-1$ puta, zatim $S \to abc$, pa "proburbla" svaki B ulijevo pokraj c-ova (pravilo $cB \to Bc$), i na kraju pretvori svaki B u b (pravilo $bB \to bb$).

Da su pravila monotona lako se provjeriti: sve desne strane su iste ili dulje od lijevih. Da gramatika generira tocno $L_\equiv$ dokazujemo **invarijantom**: rijec w u izvodu je

"balansirana" ako ima jednak broj a-ova, b-ova, B-ova i c-ova zajedno gledano.

7.4.2 Pretvorba MG $\to$ KG

Pretvorba ide u dvije faze analogno pretvorbi BKG u ChNF:

1. **TERM**: znakove koji se pojavljuju uz ostalo zamijenimo novim varijablama (i na lijevoj i na desnoj strani!).
2. **BIN**: pravila s $n \geq 2$ simbola s lijeve strane $A_1 \cdots A_n \to B_1 \cdots B_m$ (gdje $m \geq n \geq 2$) zamijenimo s $2n$ kontekstnih pravila koja "postepeno" zamjenjuju A_i s C_i, pa C_i s B_i.

7.5 Ekvivalentnost OA i MG

7.5.1 MG $\to$ OA

Za MG G, OA nedeterministicki "pogada" izvod rijeci: koristi drugi trag za meduvrijeci, primjenjuje pravila u nedeterminis- ticnom redoslijedu, i prihvaca kad se drugi trag poklopi s ulazom. Kljucno: jer su pravila monotona, meduvrijec nikad nije dulji od ulaza — dakle stane na traci.

7.5.2 OA $\to$ MG

Ovaj smjer je tehnicki slozeni ji. Koristimo **jednostavni OA** (JOA) koji prihvaca w samo u konfiguraciji $(q_X, 0, w)$ — jedinstvenoj zavrsnoj konfiguraciji.

Ideja: kodiramo konfiguracije OA kao rijeci nad $Y = Q \cup \Gamma$, jer svaka konfiguracija $(q, m, t_0 \cdots t_{n-1})$ odgovara rijeci $t_0 \cdots t_{m-1}\, q\, t_m \cdots t_{n-1}$.

Pravila gramatike simuliraju prijelaze OA:

- Za prijelaz $(q, \alpha, p, \beta, 1)$: pravilo $q\alpha \to \beta p$.
- Za prijelaz $(q, \alpha, p, \beta, -1)$: pravilo $\gamma q \alpha \to p \gamma \beta$ za svaki $\gamma \in \Gamma$.

Dodatni trik: objedinjavanje znakova ("sljepljivanje" pomocnih simbola s pravcim znakovima) osigurava da gramatika ostane monotona dok uklanjamo rubne oznake.

7.6 Zatvorenost klase Kon

Propozicija 7.6.1. *Klasa* Kon *zatvorena je na sve skupovne i jezicne operacije:* $\cup$, $\cap$, ^{c} *(komplement),* $\setminus$, $\cdot$, $^+$.

Zatvorenost na uniju, konkatenaciju i plus slijedi identicki kao za BK (dodavanjem novog pocetnog simbola).

Zatvorenost na **presjek** slijedi iz "serijskog spajanja" JOA: zavrsetno stanje prvog postaje pocetno stanje drugog. Budici da JOA cuva traku i poziciju, drugi automat pocinje tocno u stanju u kojemu je prvi zavrsio.

Zatvorenost na **komplement** je tezi:
Propozicija 7.6.2 (Immerman–Szelepcsényi, 1988)**.** *Komplement kontekstnog jezika je kontekstan.*

Ovo je netrivi jalan teorem (dokazan tek 1988., neovisno od dva autora). Ideja: nedeterministicni OA koji prepoznaje komplement koristi tehniku "brojenja" — nedeterministicno prihvaca $w \notin L$ tako sto pokaze da *ne postoji* prihvacajuca grana za w.

7.7 Usporedba klasa

	Reg	**BK**	**Kon**	**RE**
Automat	KA	PA	OA	TP
Gramatika	DLG/LLG	BKG	KG/MG	OG
Memorija	nista	stog	ogr. traka	neogr. traka
Zatvorena $\cup$	✓	✓	✓	✓
Zatvorena $\cap$	✓	×	✓	✓
Zatvorena ^{c}	✓	×	✓	×
A_L odluciv?	✓	✓	✓	samo poluodl.
E_L odluciv?	✓	✓	×	×
EQ_L odluciv?	✓	×	×	×

Zadatak 7.7.1

(a) Konstruirajte OA koji prepoznaje $\{ww^R \mid w \in \{a,b\}^*\}$.
(b) Napišite monotonu gramatiku za $\{a^n b^n c^n d^n \mid n \geq 1\}$.
(c) Dokazite da je $\{a^{2^n} \mid n \geq 0\}$ kontekstan jezik.
(d) Zasto klasa Kon nije zatvorena na Kleenevu zvijezdu? Koji je problem s ε?

7.7 Sazetak

OA	ograniceni automat: traka duljine $\|w\|$, kretanje lijevo-desno
JOA	jednostavni OA: jedinstven zavrsni uvjet
KG	kontekstna gramatika: $uAv \to ubv$
MG	monotona gramatika: $\|u\| \leq \|v\|$
OA $\equiv$ KG $\equiv$ MG	svi opisuju klasu Kon
I-S teorem	Kon zatvorena na komplement

U sljedecem poglavlju prelazimo na **Turingove strojeve** — strojeve bez ikakvog ogranicenja na traku — i pitanja odlucivosti.

Poglavlje 8

Turingovi strojevi i odlucivost

Sve sto racunala mogu raditi, mogu i Turingovi strojevi — i obrnuto. Ovo poglavlje definira taj univerzalni model izracuna, klasificira jezike na odlucive i neodlucive, te dokazuje temeljne rezultate neodlucivosti pomocu dijagonalizacije i svodenja.

8.1 Turingov stroj — motivacija

OA je mocniji od PA, ali jos uvijek ima ogranicenje: traka ne smije biti dulja od ulaza. Sto ako trebamo vise prostora?

Najjednostavnija opcija: "otvorimo" traku s desne strane. Umjesto da je traka duljine $|w|$, traka je *beskonacna* prema desno, inicijalno popunjena posebnim znakom praznine $\sqcup$.

Ovaj model zovemo **Turingov stroj** (TS), i ispostavlja se da je to upravo dovoljno za modeliranje svakog izracuna koji je ikada opisan. Ovo je sadrzaj **Church-Turingove teze**: sve sto je "algoritamski" izracunljivo, izracunljivo je i na Turingovom stroju.

TS dolazi u tri varijante s razlicitim svrhama:

Vrsta	**Svrha**	**Kljucno**
Prepoznavac (TP)	prepoznaje jezik	moze se vrtjeti unedogled
Odlucitelj (TO)	odlucuje problem	uvijek stane
Nabrajac (TE)	generira jezik	nema ulaza, nabrajamo izlaz

8.2 Turingov prepoznavac (TP)

Definicija 8.2.1 — Turingov prepoznavac

Turingov prepoznavac nad Σ je sedmorka

$$T = (Q,\ \Sigma,\ \Gamma,\ \sqcup,\ \delta,\ q_0,\ q_X),$$

gdje je $\Sigma \subsetneq \Gamma$ (radna abeceda je sira od ulazne), $\sqcup \in \Gamma \setminus \Sigma$ je znak praznine, a

$$\delta : (Q \setminus \{q_X\}) \times \Gamma \to Q \times \Gamma \times \{-1, 1\}$$

je **deterministicna** funkcija prijelaza.
Konfiguracija je trojka $(q, n, t) \in Q \times \mathbb{N} \times \Gamma_{\sqcup}^{\mathbb{N}}$, gdje je $\Gamma_{\sqcup}^{\mathbb{N}}$ skup svih traka koje su od nekog mjesta nadalje konstanta $\sqcup$ (prebrojivo mnogo takvih).
T prihvaca rijec w ako izracunavanje pocevsi od $(q_0, 0, w\sqcup\sqcup\cdots)$ dosegne konfiguraciju sa stanjem q_X.

Definicija 8.2.2 — Rekurzivno prebrojivi jezici

Jezik je **rekurzivno prebrojiv** (u klasi RE) ako ga prepoznaje neki TP. Pise se $L \in \mathrm{RE}$.

Kljucna razlika od OA: TP se *ne mora zaustaviti* za rijeci koje ne prihvaca. Moze se vrtjeti unedogled. To je problem: ne znamo je li stroj "razmislja" ili je u beskonacnoj petlji.

8.3 Turingov odlucitelj (TO)

Definicija 8.3.1 — Turingov odlucitelj i rekurzivni jezici

Turingov odlucitelj (TO) je TP koji uz zavrsno stanje q_X ima i stanje odbijanja $\overline{q_X}$, te se **uvijek** zaustavi — ili u q_X (prihvacanje) ili u $\overline{q_X}$ (odbijanje).
Jezik je **rekurzivan** (u klasi Rek) ako ga prepoznaje neki TO.

Svaki TO je TP (možemo ignorirati $\overline{q_X}$), pa $\mathrm{Rek} \subseteq \mathrm{RE}$.
Propozicija 8.3.1 (Postov teorem)**.** $L \in \mathrm{Rek}$ *ako i samo ako su i* L *i* L^c *u* RE.

Dokaz. ($\Rightarrow$): ako je L rekurzivan, i L i L^c su rekurzivni (Rek je zatvorena na komplement), pa a fortiori u RE.

($\Leftarrow$): neka TP S prepoznaje L, a TP T prepoznaje L^c. Konstruiramo TO koji istovremeno simulira S i T, korak po korak. Jer je svaka rijec ili u L ili u L^c, jedan od njih ce sigurno stati. Kad S prihvati, TO prihvaca; kad T prihvati, TO odbija. $\square$

Ključna ideja

Postov teorem znaci: jezik je odluciv tocno onda kad mozemo i *potvrditi* i *zanijekati* clanstvo algoritmom koji uvijek stane. Samo znati "kako naci" nije dovoljno — treba znati i "kako ne naci".

8.4 Zatvorenost klasa Rek i RE

Propozicija 8.4.1. *Klasa* Rek *zatvorena je na sve skupovne i jezicne operacije. Klasa* RE *zatvorena je na* $\cup$ *i* $\cap$*, ali* ***ne*** *na komplement.*

Kljucna tehnika za Rek: **potpuna simulacija**. Naredba "simuliraj rad od T na sadrzaju trake t_i" pokrece TO na kopiji trake, te uvijek stane jer je T odlucitelj.

Primjer — zatvorenost Rek na Kleenevu zvijezdu: za rijec w, probamo sve moguce rastave $w = w_1 \cdots w_k$ (konacno mnogo) i za svaki rastav provjerimo je li svaki $w_i \in L$. Ako ikoji rastav uspije, prihvati; inace odbij.

Za RE s unijom: moramo simulirati oba TP *naizmjenicno*, korak po korak — ne smijemo pokrenuti prvi do kraja jer se mozda nikad ne zaustavi. Ovo je tzv. "dovetailing" tehnika.

8.5 Kodiranje i problemi

Da bismo mogli govoriti o TS-ovima kao ulazima TS-u, trebamo ih **kodirati** kao rijeci. Koristimo **univerzalnu abecedu** $\Sigma = \{., \mathrm{I}, (,)\}$ i sustavno kodiramo:

$$c(n) := \mathrm{I}^n., \quad c(t_1, \ldots, t_k) := (c(t_1) \cdots c(t_k)), \quad c(\{t_1, \ldots, t_k\}) := (c(t_1) \cdots c(t_k)).$$

Kod stroja T (ili drugog matematickog objekta) oznacavamo $\langle T \rangle$, a par (T, w) oznacavamo $\langle T, w \rangle$.

8.5.1 Prirodni strojevi i problemi

Problem je jezik cija konkretna kodna abeceda nije bitna — bitna je samo "intencija" (sto reprezentira). Primjer:

$$A_{\mathrm{KA}} := \{\langle M, w \rangle : M \text{ je KA i } w \in L(M)\}$$

je odluciv problem (pisemo pseudokodom koji simulira KA M na w).

8.6 Odlucivi i neodlucivi problemi

8.6.1 Odlucivi problemi

Propozicija 8.6.1. *Sljedeci problemi su odlucivi:*

- A_{KA}, A_{RI}, A_{NKA} — *prihvacanje za regularne jezike*
- E_{KA} — *praznost KA-jezika*
- $\mathrm{EQ}_{\mathrm{KA}}$ — *jednakost dva KA-jezika*
- A_{BKG}, A_{BK} — *prihvacanje za BK-jezike (CYK!)*
- E_{BKG}, E_{BK} — *praznost BK-jezika*
- A_{OA}, A_{Kon} — *prihvacanje za kontekstne jezike*

Svi ovi problemi imaju konkretne algoritme koje smo vec opisali (pseudokodom ili formalnom konstrukcijom).

8.6.2 Neodlucivi problemi — hijerarhija

Problem	Klasa	Zasto
A_{TP}, A_{RE}	$\mathrm{RE} \setminus \mathrm{Rek}$	TP moze sebe simulirati
$\overline{A_{\mathrm{TP}}}$	nije u RE	Postov teorem + $A_{\mathrm{TP}} \in \mathrm{RE} \setminus \mathrm{Rek}$
$\mathrm{EQ}_{\mathrm{BK}}$	neodlucivo	svodenje od $\overline{A_{\mathrm{TP}}}$
E_{Kon}	neodlucivo	svodenje od A_{TP}

8.7 Svodenje (redukcija)

Definicija 8.7.1 — Svodenje

Kažemo da se L **svodi na** M ($L \leq M$) ako postoji totalna Turing-izracunljiva funkcija $\varphi : \Sigma^* \to \Sigma^*$ takva da za svaki w:

$$w \in L \iff \varphi(w) \in M.$$

Propozicija 8.7.1. *Neka $L \leq M$.*

- *Ako je M (polu)odluciv, tada je i L (polu)odluciv.*
- *Ako je L neodluciv, tada je i M neodluciv.*

Drugu tocku koristimo za dokazivanje neodlucivosti: pokazemo POZNATO_NEODLUCIVO $\leq$ NOVI_PROBLEM, cime zakljucujemo da je i novi problem neodluciv.

Primjer 8.7.1 — Svodenje za E_{KA}

Problem jednakosti $\mathrm{EQ}_{\mathrm{KA}}$ svodi se na E_{KA}:

$$\langle M_1, M_2 \rangle \mapsto \langle M_\Delta \rangle, \quad \text{gdje } L(M_\Delta) = L(M_1) \triangle L(M_2).$$

Dakle $\langle M_1, M_2 \rangle \in \mathrm{EQ}_{\mathrm{KA}} \iff \langle M_\Delta \rangle \in E_{\mathrm{KA}}$. Konstruisanje M_Δ je mehanicko (Kartezijev produkt), pa je φ Turing-izracunljiva. Kako je E_{KA} odluciv, slijedi da

je i $\mathrm{EQ}_{\mathrm{KA}}$ odluciv. ✓

8.8 Dijagonalizacija: Russell i A_{TP}

Kako dokazati da neki jezik *nije* u RE? Lema o pumpanju ne postoji za ovu klasu. Koristimo **dijagonalizaciju** — trik koji potjece od Cantora (prebrojiivost skupova) i Russella (paradoks).

Propozicija 8.8.1. *Jezik*

$$\text{Russell} := \{\langle T\rangle : T \textit{ je TP i } \langle T\rangle \notin L(T)\}$$

nije rekurzivno prebrojiv.

Dokaz. Pretpostavimo da je Russell u RE, i neka TP R prepoznaje Russell. Tada:

$$\langle R\rangle \in \text{Russell} \iff \langle R\rangle \notin L(R) = \text{Russell}.$$

Kontradikcija. □

Ovo je direktan analog Russellovog paradoksa: "skup svih skupova koji ne sadrze sami sebe" vodi do kontradikcije.

Propozicija 8.8.2. *Jezik* $A_{\mathrm{TP}} = \{\langle T, w\rangle : T \textit{ prihvaca } w\}$ *je u* $\mathrm{RE} \setminus \mathrm{Rek}$.

Skica dokaza. $A_{\mathrm{TP}} \in \mathrm{RE}$: konstruiramo **univerzalni Turingov stroj** (UTS) koji simulira rad TP T na ulazu w (dan kao kod). UTS je sam po sebi TP, pa je $A_{\mathrm{TP}} \in \mathrm{RE}$.

$A_{\mathrm{TP}} \notin \mathrm{Rek}$: dokazemo da $\overline{A_{\mathrm{TP}}}$ nije u RE. Koristimo Russell $\leq \overline{A_{\mathrm{TP}}}$: svaki kod $\langle T\rangle$ preslikamo u $\langle T, \langle T\rangle\rangle$, i vrijedi:

$$\langle T\rangle \in \text{Russell} \iff \langle T\rangle \notin L(T) \iff \langle T, \langle T\rangle\rangle \in \overline{A_{\mathrm{TP}}}.$$

Dakle Russell $\leq \overline{A_{\mathrm{TP}}}$. Budici da Russell nije u RE, ni $\overline{A_{\mathrm{TP}}}$ nije u RE. Po Postovom teoremu, $A_{\mathrm{TP}} \notin \mathrm{Rek}$. □

8.9 Problem zaustavljanja i kompletna slika

Propozicija 8.9.1. *Jezik* $H = \{\langle T, w\rangle : \textit{izracunavanje } T \textit{ na } w \textit{ stane}\}$ *("halting problem") ekvivalentan je* A_{TP} *u smislu svedivosti:* $H \leq A_{\mathrm{TP}}$ *i* $A_{\mathrm{TP}} \leq H$.

Ova ekvivalencija nije iznenadujuca: TP koji smo definirali stane tocno kada prihvate, pa "stajanje" i "prihvacanje" su isti uvjet.

8.9.1 Kompletna Chomskyjeva hijerarhija

Klasa	**Automat**	**Gram.**	A_L	E_L
Reg	KA	DLG	odluciv	odluciv
BK	PA	BKG	odluciv	odluciv
Kon	OA	KG/MG	odluciv	neodluciv
Rek	TO	—	odluciv	neodluciv
RE	TP	OG	poluodluciv	neodluciv

Prave inkluzije potvrdjuju konkretni jezici:

$$\begin{aligned} L_= &= \{a^n b^n\} \in \mathrm{BK} \setminus \mathrm{Reg} \\ L_\equiv &= \{a^n b^n c^n\} \in \mathrm{Kon} \setminus \mathrm{BK} \\ \mathrm{EQ}_{\mathrm{RI}_2} &\in \mathrm{Rek} \setminus \mathrm{Kon} \\ A_{\mathrm{TP}} &\in \mathrm{RE} \setminus \mathrm{Rek} \\ \overline{A_{\mathrm{TP}}} &\notin \mathrm{RE} \end{aligned}$$

8.10 Nedeterministicki Turingov stroj (NTP)

Definicija 8.10.1

NTP ima relaciju prijelaza umjesto funkcije. Prihvaca rijec ako *postoji barem jedna* grana izracunavanja koja stane.

Propozicija 8.10.1. *NTP i TP su ekvivalentni: svaki NTP ima ekvivalentni TP.*

Dokaz: TP simulira NTP koristeci **tri trake**:

1. Kopija ulaza (nepromjenjiva).
2. Radna traka (simulacija jedne grane).
3. Trecina traka: kôd trenutne grane kao niz odabranih prijelaza iz Δ^* (leksikografskim redoslijedom isprobavamo sve grane).

TP prolazi kroz sve moguce grane BFS-om: svaka iteracija produlji granu za jedan korak. Cim neka grana prihvati, TP prihvaca.

Napomena. *Za odlucitelje (NTO) analogna ekvivalencija vrijedi, ali je dokaz tezi — ne moze koristiti istu trotracnu simulaciju jer ne znamo znaci li $\overline{q_X}$ konacno odbijanje ili samo lok alni neuspjeh.*

8.11 Turingovi nabrajaci (TE)

Definicija 8.11.1 — Turingov nabrajac

TE nema ulaza. Ima radnu traku i **izlaznu traku**, te posebno **izlazno stanje** $q_{\leftarrow}$. Rad TE je uvijek beskonacni niz konfiguracija. TE **izvodi** rijec w ako ikad dosegne stanje $q_{\leftarrow}$ s w na izlaznoj traci.
$L(E) := \{w : E \text{ izvodi } w\}$.

Propozicija 8.11.1. $L \in \mathrm{RE}$ *ako i samo ako postoji TE koji generira* L.

Smjer ($\Leftarrow$): TP koji prepoznaje $L(E)$ simulira E korak po korak i usporeduje izlaz s ulaznom rijeci w. Cim se poklope, prihvaca.

Smjer ($\Rightarrow$): TE za L (prepoznajan TP T) nabrajanjem: za svaki n i svaki moguc ulaz duljine $\leq n$, simulira T na tom ulazu tocno n koraka. Ako T prihvati, izvede tu rijec. Ovim "dovetailing" postupkom svaka rijec iz L ce biti izvedena u konacno mnogo koraka.

8.11 Sazetak

TP	Turingov prepoznavac: stane ili prihvati (nema garantije zaustavljanja)
TO	Turingov odlucitelj: uvijek stane, prihvati ili odbij
RE	jezici koje prepoznaje neki TP
Rek	jezici koje prepoznaje neki TO
A_{TP}	poluodluciv ali neodluciv (svodenje + dijagonalizacija)
Postov teorem	$L \in \mathrm{Rek} \iff L, L^c \in \mathrm{RE}$
Svodenje	$L \leq M$: odlucivost "prenosi se gore"

$$\mathrm{Reg} \subsetneq \mathrm{BK} \subsetneq \mathrm{Kon} \subsetneq \mathrm{Rek} \subsetneq \mathrm{RE} \subsetneq \textit{svi jezici}$$